COSC2793 Computational Machine Learning — Assignment 3

Histopathology Colon-Cell Image Classification

This notebook follows the report structure (sections 0–J). Two prediction tasks:

- **Task A — `isCancerous`** (binary): uses `mainData + extraData`
- **Task B — `cellType`** (4-class): uses `mainData` only

  Targets: Task A ≥ 0.90, Task B ≥ 0.60 on unseen patients.

0. Introduction & Setup

In [1]:
```
import os
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# All data lives under ./data (images/ subfolder + two label CSVs).
DATA_DIR = "data"
IMG_DIR  = os.path.join(DATA_DIR, "images")

print("Setup complete. Data dir exists:", os.path.isdir(IMG_DIR))
```

```
Setup complete. Data dir exists: True
```

In [ ]:
```
# PLOTS_DIR = "plots"
# os.makedirs(PLOTS_DIR, exist_ok=True)

# def save_fig(fig, name, dpi=150):
#     """Save a matplotlib figure to ./plots/<name>.png at report quality.
#     Call this BEFORE plt.show()."""
#     path = os.path.join(PLOTS_DIR, f"{name}.png")
#     fig.savefig(path, dpi=dpi, bbox_inches="tight")
#     print(f"[saved] {path}")
```

A. Task Definition

This project addresses two related supervised image-classification problems on 27×27 RGB histopathology images of colon cells drawn from the CRCHistoPhenotypes dataset:

- **Task A — Cancer detection (`isCancerous`).** A classification predicts whether a cell image depicts cancerous tissue (1) or not (0). Labels are available for all 99 patients, so this task uses both `data_labels_mainData.csv` and `data_labels_extraData.csv` .

- **Task B — Cell-type identification (`cellType`).** A *multi-class* classification assign each cell to one of four phenotypes: *epithelial, inflammatory, fibroblast,* or *others*. Cell-type labels were provided by medical experts for the 60 patients in `data_labels_mainData.csv` only, so this task is restricted to that file.

B. Dataset Description & Exploratory Data Analysis (EDA)

In [3]:
```
# Step 1: load label files and construct one dataframe per task
main_df  = pd.read_csv(os.path.join(DATA_DIR, "data_labels_mainData.csv"))
extra_df = pd.read_csv(os.path.join(DATA_DIR, "data_labels_extraData.csv"))

# --- Task A (isCancerous): both files. ---
# The two files share four columns but extra_df lacks the cell-type columns,
# so we align on the common schema before concatenating.
common_cols = ["InstanceID", "patientID", "ImageName", "isCancerous"]
dfA = pd.concat(
    [main_df[common_cols], extra_df[common_cols]],
    ignore_index=True,
)

# --- Task B (cellType): main file only (the only one with expert cell-type l
dfB = main_df.copy()



# --- Sanity checks / EDA inputs ---
print("Task A (isCancerous) ----------------------------------")
print("shape:", dfA.shape, "| patients:", dfA["patientID"].nunique())
print(dfA["isCancerous"].value_counts().to_dict())
print()
print("Task B (cellType) -------------------------------------")
print("shape:", dfB.shape, "| patients:", dfB["patientID"].nunique())
print(dfB["cellTypeName"].value_counts().to_dict())
```

```
Task A (isCancerous) ----------------------------------
shape: (20280, 4) | patients: 98
{0: 13211, 1: 7069}

Task B (cellType) -------------------------------------
shape: (9896, 6) | patients: 60
{'epithelial': 4079, 'inflammatory': 2543, 'fibroblast': 1888, 'others': 138
6}
```

In [4]:
```
# Step 2: load image pixels into arrays
def load_images(df, img_dir=IMG_DIR):
    """Read every image referenced in df['ImageName'] in row order and
    return them as a single uint8 array of shape (N, 27, 27, 3)."""
    imgs = np.empty((len(df), 27, 27, 3), dtype=np.uint8)
    for i, name in enumerate(df["ImageName"].values):
        with Image.open(os.path.join(img_dir, name)) as im:
            imgs[i] = np.asarray(im.convert("RGB"))
    return imgs

# Load pixels for each task, kept in the SAME row order as dfA / dfB so that
# image XA[i] corresponds exactly to label row dfA.iloc[i].
XA = load_images(dfA)              # Task A: all 20,280 images
yA = dfA["isCancerous"].values     # binary labels {0,1}

XB = load_images(dfB)              # Task B: the 9,896 main-data images
```

```
yB = dfB["cellType"].values        # integer cell-type labels {0,1,2,3}

print("Task A images:", XA.shape, XA.dtype, "| labels:", yA.shape)
print("Task B images:", XB.shape, XB.dtype, "| labels:", yB.shape)
print("Pixel value range:", XA.min(), "-", XA.max())
print("Memory (XA): {:.1f} MB".format(XA.nbytes / 1e6))
```

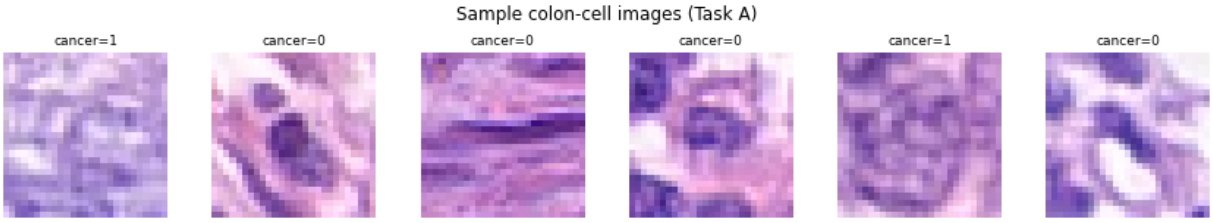
```
Task A images: (20280, 27, 27, 3) uint8 | labels: (20280,)
Task B images: (9896, 27, 27, 3) uint8 | labels: (9896,)
Pixel value range: 0 - 255
Memory (XA): 44.4 MB
```

Dataset characteristics

- 20,280 RGB images, 27×27 px, from 99 patients. Task A (`isCancerous`) uses all 20,280 images; Task B (`cellType`) uses the 9,896 main-data images.
- Images has low resolution and are visually noisy, so each cell carries limited discriminative information.
- Images are grouped by patient, so generalisation must be assessed on unseen patients.

In [5]:

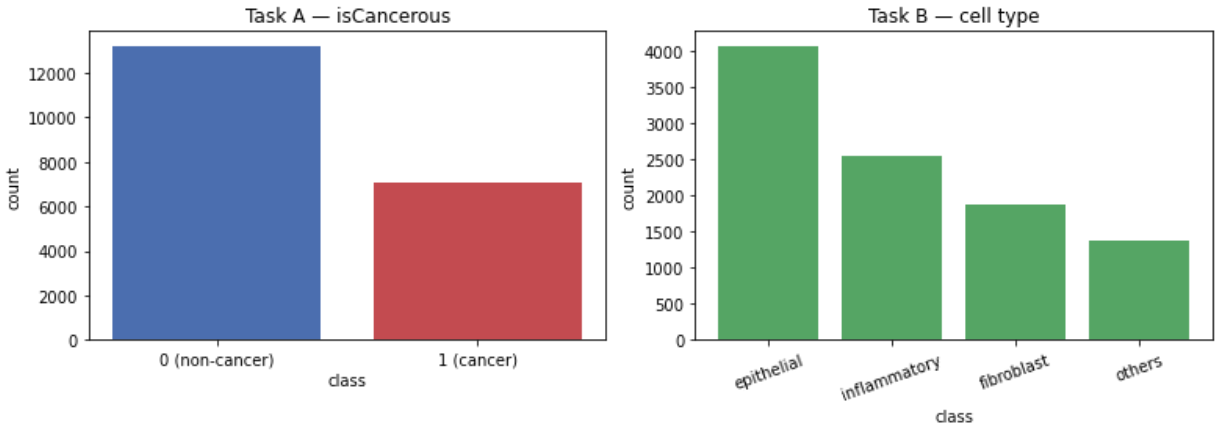
```
# Quick visual check: a few images with their labels
fig, axes = plt.subplots(1, 6, figsize=(12, 2.2))
for ax, idx in zip(axes, np.random.choice(len(XA), 6, replace=False)):
    ax.imshow(XA[idx])
    ax.set_title(f"cancer={yA[idx]}", fontsize=9)
    ax.axis("off")
plt.suptitle("Sample colon-cell images (Task A)")
plt.tight_layout(); plt.show()
```



In [ ]:

```
# Step 3: class distribution for both tasks
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

vcA = dfA["isCancerous"].value_counts().sort_index()
axes[0].bar(["0 (non-cancer)", "1 (cancer)"], vcA.values,
            color=["#4c72b0", "#c44e52"])
axes[0].set_title("Task A — isCancerous")
axes[0].set_xlabel("class"); axes[0].set_ylabel("count")

vcB = dfB["cellTypeName"].value_counts()
axes[1].bar(vcB.index, vcB.values, color="#55a868")
axes[1].set_title("Task B — cell type")
axes[1].set_xlabel("class"); axes[1].set_ylabel("count")
axes[1].tick_params(axis="x", rotation=20)

plt.tight_layout()
plt.show()

print("Task A class proportions:")
print(dfA["isCancerous"].value_counts(normalize=True).round(3).to_dict())
print("Task B class proportions:")
print(dfB["cellTypeName"].value_counts(normalize=True).round(3).to_dict())
```

```
[saved] plots/eda_class_distribution.png
```




```
Task A class proportions:
{0: 0.651, 1: 0.349}
Task B class proportions:
{'epithelial': 0.412, 'inflammatory': 0.257, 'fibroblast': 0.191, 'others':
0.14}
```

Class imbalance

- Task A: ~65% non-cancer / 35% cancer (≈1.9:1).
- Task B: epithelial 41%, inflammatory 26%, fibroblast 19%, others 14%.
- The data in each task is unbalanced, so accuracy is misleading. The primary metric is **macro-F1** because it weights equally per class.

In [ ]:

```
# Step 4: mean image per class / example grids
celltype_map = (dfB.drop_duplicates("cellType")
                   .set_index("cellType")["cellTypeName"].to_dict())
print("cellType mapping:", celltype_map)

def show_mean_images(X, labels, names, title, save_name=None):
    classes = sorted(np.unique(labels))
    fig, axes = plt.subplots(1, len(classes), figsize=(3 * len(classes), 3))
    if len(classes) == 1:
        axes = [axes]
    for ax, c in zip(axes, classes):
        mean_img = X[labels == c].mean(axis=0).astype(np.uint8)
        ax.imshow(mean_img)
        ax.set_title(f"{names[c]}\n(n={np.sum(labels == c)})", fontsize=9)
        ax.axis("off")
    plt.suptitle(title); plt.tight_layout()

    plt.show()

show_mean_images(XA, yA, {0: "non-cancer", 1: "cancer"},
                 "Task A — mean image per class", "eda_mean_taskA")
show_mean_images(XB, yB, celltype_map,
                 "Task B — mean image per class", "eda_mean_taskB")
```

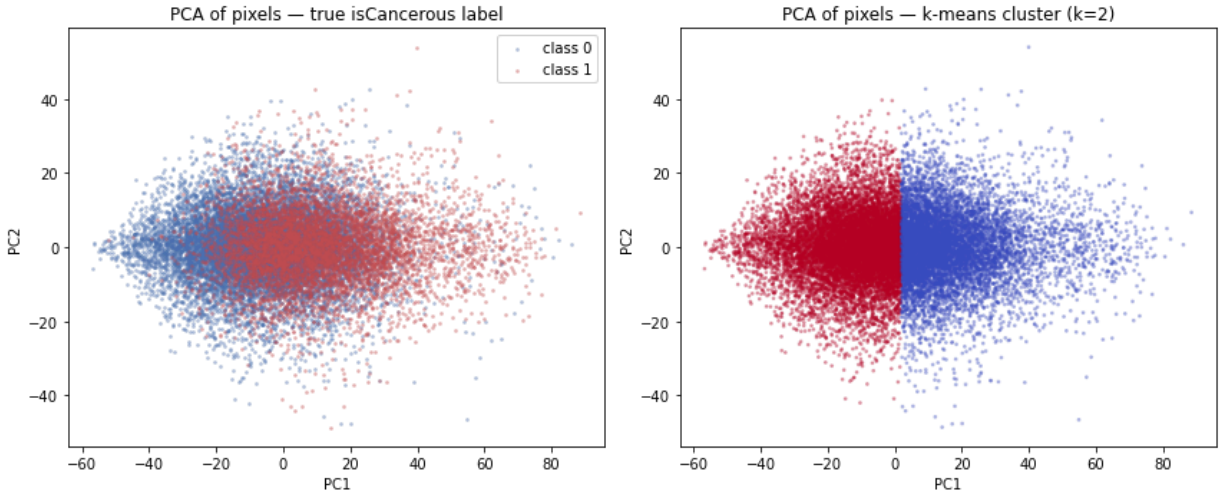
```
cellType mapping: {0: 'fibroblast', 1: 'inflammatory', 3: 'others', 2: 'epith
elial'}
[saved] plots/eda_mean_taskA.png
```


```
[saved] plots/eda_mean_taskB.png
```

In [ ]:

```
# Step 5: unsupervised EDA (PCA + KMeans on flattened pixels)
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

XA_flat = XA.reshape(len(XA), -1).astype(np.float32)   # (N, 2187)
Xs = StandardScaler().fit_transform(XA_flat)
pca = PCA(n_components=2, random_state=RANDOM_STATE)
proj = pca.fit_transform(Xs)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
for c, col in zip([0, 1], ["#4c72b0", "#c44e52"]):
    m = yA == c
    ax[0].scatter(proj[m, 0], proj[m, 1], s=3, alpha=0.3, color=col, label=f"
ax[0].set_title("PCA of pixels — true isCancerous label")
ax[0].set_xlabel("PC1"); ax[0].set_ylabel("PC2"); ax[0].legend()

km = KMeans(n_clusters=2, n_init=10, random_state=RANDOM_STATE).fit(proj)
ax[1].scatter(proj[:, 0], proj[:, 1], s=3, alpha=0.3, c=km.labels_, cmap="coo
ax[1].set_title("PCA of pixels — k-means cluster (k=2)")
ax[1].set_xlabel("PC1"); ax[1].set_ylabel("PC2")

plt.tight_layout()
plt.show()

print("Variance explained by PC1, PC2:", pca.explained_variance_ratio_.round(
print(f"Adjusted Rand Index (k-means vs true labels): "
    f"{adjusted_rand_score(yA, km.labels_):.3f}")
```

```
[saved] plots/eda_pca_kmeans.png
```




```
Variance explained by PC1, PC2: [0.184 0.042]
Adjusted Rand Index (k-means vs true labels): 0.115
```

Unsupervised structure (PCA + k-means)

- PC1 and PC2 explain only 18.4% and 4.2% of pixel variance (~22.6% combined) — variation is spread across many directions.
- K-means (k=2) on the PCA projection scores ARI 0.115 versus true labels (near chance) and splits on PC1 (staining intensity), not cancer status.
- Classes are not linearly separable in raw pixel space. So, it motivates to explore the new models.

C. Data Pre-processing

Patient-level data splitting

- Images are grouped by patient, and the task measures how the best model generalise the unseen patients. A random train_test_split would put one patient's cells in both train and test. Since a patient's cells share staining, illumination and tissue traits, the model could exploit those cues, which leads group-level leakage.
- So we split by patient with GroupShuffleSplit on patientID, keeping each patient entirely within one split: 60% train / 20% val / 20% test.
- Test patients are held out and never used for any development decision. It will be used at the end.
- We tune hyperparameter in this assingment too.
- Because GroupShuffleSplit cannot also balance classes, we verify after splitting that patient sets are disjoint and class proportions are roughly preserved.

In [10]:

```
# Step 6: patient-level split (no patient overlap)
from sklearn.model_selection import GroupShuffleSplit

def patient_group_split(df, test_size=0.20, val_frac_of_remainder=0.25,
                        random_state=RANDOM_STATE):
    """Return disjoint (train, val, test) row indices, split by patientID.
    test_size=0.20 -> 20% test; val_frac_of_remainder=0.25 of the remaining 8
    -> 20% val; leaving 60% train. No patient appears in more than one split.
    groups = df["patientID"].values
    idx = np.arange(len(df))

    # 1) hold out the test patients
```

```
    gss_test = GroupShuffleSplit(n_splits=1, test_size=test_size,
                                 random_state=random_state)
    trainval_idx, test_idx = next(gss_test.split(idx, groups=groups))

    # 2) carve validation patients out of the remaining train+val patients
    gss_val = GroupShuffleSplit(n_splits=1, test_size=val_frac_of_remainder,
                                random_state=random_state)
    tr_rel, val_rel = next(gss_val.split(trainval_idx,
                                         groups=groups[trainval_idx]))
    train_idx = trainval_idx[tr_rel]
    val_idx   = trainval_idx[val_rel]
    return train_idx, val_idx, test_idx


def report_split(df, train_idx, val_idx, test_idx, label_col, task_name):
    """Verify disjoint patients and show size + class balance per split."""
    p = df["patientID"].values
    s_tr, s_va, s_te = set(p[train_idx]), set(p[val_idx]), set(p[test_idx])
    print(f"=== {task_name} ===")
    print(f"images  -> train {len(train_idx)} | val {len(val_idx)} | test {le
    print(f"patients-> train {len(s_tr)} | val {len(s_va)} | test {len(s_te)}
    print("disjoint patients (no overlap):",
        s_tr.isdisjoint(s_va) and s_tr.isdisjoint(s_te) and s_va.isdisjoint(s
    for name, ix in [("train", train_idx), ("val", val_idx), ("test", test_id
        props = df.iloc[ix][label_col].value_counts(normalize=True).round(3).
        print(f"  {name} class balance: {props.to_dict()}")
    print()
```

In [11]:

```
# --- Task A split ---
trainA_idx, valA_idx, testA_idx = patient_group_split(dfA)
report_split(dfA, trainA_idx, valA_idx, testA_idx, "isCancerous", "Task A (is
```

```
=== Task A (isCancerous) ===
images  -> train 12416 | val 3908 | test 3956
patients-> train 58 | val 20 | test 20
disjoint patients (no overlap): True
  train class balance: {0: 0.634, 1: 0.366}
  val class balance: {0: 0.621, 1: 0.379}
  test class balance: {0: 0.735, 1: 0.265}
```

In [12]:

```
# --- Task B split ---
trainB_idx, valB_idx, testB_idx = patient_group_split(dfB)
report_split(dfB, trainB_idx, valB_idx, testB_idx, "cellTypeName", "Task B (c
```

```
=== Task B (cellType) ===
images  -> train 5671 | val 2386 | test 1839
patients-> train 36 | val 12 | test 12
disjoint patients (no overlap): True
  train class balance: {'epithelial': 0.392, 'fibroblast': 0.233, 'inflammato
ry': 0.248, 'others': 0.127}
  val class balance: {'epithelial': 0.39, 'fibroblast': 0.127, 'inflammator
y': 0.296, 'others': 0.187}
  test class balance: {'epithelial': 0.503, 'fibroblast': 0.142, 'inflammator
y': 0.235, 'others': 0.12}
```

Feature scaling

- Pixels lie in [0, 255]. Distance-based and gradient-based models (SVM, logistic regression, neural nets) train stably with standardisation.

- Tree ensembles are scale-invariant and unaffected. Scaling statistics are fitted on the training split only, then we applied to validation and test because fitting on the full dataset would leak validation/test patient information into the transformatio. Therefore, so train-only fitting keeps the evaluation honest.
- From the same split we build two representations: a flattened standardised vector of length 2,187 for the classical (non-spatial) models, and the 27×27×3 per-channel-normalised image tensor for the CNN, which needs the spatial layout preserved.

In [14]:

```
# Step 7: build scaled representations, fitting statistics on TRAIN ONLY.
from sklearn.preprocessing import StandardScaler
# (a) Flattened + standardised vectors for classical models (SVM, logreg, tre
def make_flat_splits(X, tr_idx, va_idx, te_idx):
    Xf = X.reshape(len(X), -1).astype(np.float32)      # (N, 2187), raw 0-255
    scaler = StandardScaler().fit(Xf[tr_idx])          # fit on TRAIN only
    return (scaler.transform(Xf[tr_idx]),
            scaler.transform(Xf[va_idx]),
            scaler.transform(Xf[te_idx]),
            scaler)

XA_tr, XA_va, XA_te, scalerA = make_flat_splits(XA, trainA_idx, valA_idx, tes
yA_tr, yA_va, yA_te = yA[trainA_idx], yA[valA_idx], yA[testA_idx]

XB_tr, XB_va, XB_te, scalerB = make_flat_splits(XB, trainB_idx, valB_idx, tes
yB_tr, yB_va, yB_te = yB[trainB_idx], yB[valB_idx], yB[testB_idx]

print("Flat (classical-model) representations:")
print("  Task A:", XA_tr.shape, XA_va.shape, XA_te.shape)
print("  Task B:", XB_tr.shape, XB_va.shape, XB_te.shape)

# (b) Per-channel mean/std from TRAIN images, for the CNN later (kept as cons
train_imgs_A = XA[trainA_idx].astype(np.float32) / 255.0     # scale to [0,1]
CH_MEAN = train_imgs_A.mean(axis=(0, 1, 2))                  # one value per
CH_STD  = train_imgs_A.std(axis=(0, 1, 2))
print("\nTask A train per-channel mean:", CH_MEAN.round(4), "std:", CH_STD.ro
```

```
Flat (classical-model) representations:
  Task A: (12416, 2187) (3908, 2187) (3956, 2187)
  Task B: (5671, 2187) (2386, 2187) (1839, 2187)

Task A train per-channel mean: [0.7695 0.6016 0.8493] std: [0.1681 0.1895 0.1
175]
```

D. Model Development & Performance Analysis

- ≥3 supervised models **per task**. For each: how it works, why chosen, pros/cons. For ≥1 model: vary a hyperparameter with curves; analyse under/overfitting, convergence.
- We build **Task A end-to-end first**, then reuse the pipeline for Task B.

- For each task we develop three models of increasing capacity: a **Logistic Regression** (linear baseline on flattened pixels), a **Decision Tree** (non-linear, scale-invariant baseline), and a **Convolutional Neural Network** (exploits spatial structure).
- All models are trained on the identical patient-level split and evaluated with the same harnes. The primary metric **macro-F1** would be reported on the validation set. The test set is reserved for final evaluation.

In [ ]:

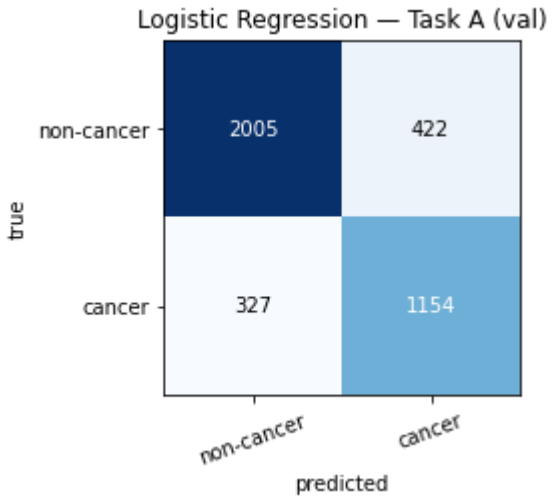
```
# Shared evaluation harness — identical scoring for every model.
from sklearn.metrics import (accuracy_score, f1_score,
                             classification_report, confusion_matrix)

results = []  # accumulates one row per (task, model) for the Section F compa

def evaluate(model_name, task, y_true, y_pred, class_names,
             split="val", save=None):
    acc = accuracy_score(y_true, y_pred)
    mf1 = f1_score(y_true, y_pred, average="macro")
    results.append({"task": task, "model": model_name, "split": split,
                    "accuracy": round(acc, 4), "macro_f1": round(mf1, 4)})
    print(f"[Task {task}] {model_name} ({split}): "
        f"accuracy={acc:.3f} | macro-F1={mf1:.3f}")
    print(classification_report(y_true, y_pred, target_names=class_names, dig

    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(0.9 * len(class_names) + 2,
                                    0.9 * len(class_names) + 2))
    im = ax.imshow(cm, cmap="Blues")
    ax.set_xticks(range(len(class_names))); ax.set_yticks(range(len(class_nam
    ax.set_xticklabels(class_names, rotation=20); ax.set_yticklabels(class_na
    ax.set_xlabel("predicted"); ax.set_ylabel("true")
    ax.set_title(f"{model_name} — Task {task} ({split})")
    thr = cm.max() / 2
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, cm[i, j], ha="center", va="center",
                    color="white" if cm[i, j] > thr else "black")
    plt.tight_layout()

    plt.show()
    return mf1
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, cm[i, j], ha="center", va="center",
                    color="white" if cm[i, j] > thr else "black")
    plt.tight_layout()

    plt.show()
    return mf1
```

D.1 Task A — Model 1: (non-NN baseline, e.g. Decision Tree / SVM)

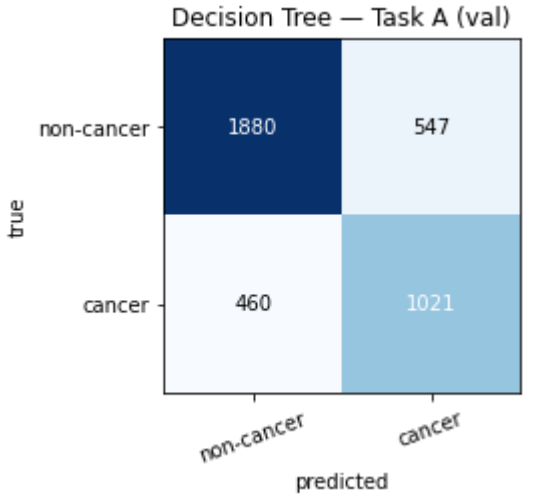
In [16]:

```
# D.1: classical baselines on flattened, standardised pixels
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

A_names = ["non-cancer", "cancer"]

# --- Logistic Regression (linear) ---
logregA = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
logregA.fit(XA_tr, yA_tr)
evaluate("Logistic Regression", "A", yA_va, logregA.predict(XA_va),
        A_names, save="cm_A_logreg")

# --- Decision Tree (non-linear) ---
treeA = DecisionTreeClassifier(random_state=RANDOM_STATE)
treeA.fit(XA_tr, yA_tr)
evaluate("Decision Tree", "A", yA_va, treeA.predict(XA_va),
        A_names, save="cm_A_tree")
```

```
/Users/hieutong/opt/anaconda3/lib/python3.8/site-packages/sklearn/linear_mode
l/_logistic.py:460: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres
sion
  n_iter_i = _check_optimize_result(
[Task A] Logistic Regression (val): accuracy=0.808 | macro-F1=0.799
              precision    recall  f1-score   support

  non-cancer      0.860     0.826     0.843      2427
      cancer      0.732     0.779     0.755      1481

    accuracy                          0.808      3908
   macro avg      0.796     0.803     0.799      3908
weighted avg      0.811     0.808     0.809      3908

[saved] plots/cm_A_logreg.png
```




```
[Task A] Decision Tree (val): accuracy=0.742 | macro-F1=0.729
              precision    recall  f1-score   support

  non-cancer      0.803     0.775     0.789      2427
      cancer      0.651     0.689     0.670      1481

    accuracy                          0.742      3908
   macro avg      0.727     0.732     0.729      3908
weighted avg      0.746     0.742     0.744      3908

[saved] plots/cm_A_tree.png
```



Out[16]: 0.7292419053233243

- Both are well below the 0.90 target.
- The simpler linear (logistic regression) model perform better than the more complex non-linear tree (0.799 vs 0.729). A single unconstrained Decision Tree grows until it perfectly fits the training data, including the noise in 2,187 pixel features → it overfits → high variance → worse generalisation. Logistic Regression's linear form acts as built-in regularisation.
- Cancer (the minority, clinically critical class) is harder for both — F1 of 0.755 / 0.670 vs ~0.84/0.79 for non-cancer. The model misses more cancer cells (false negatives), which is the dangerous error type.

D.2 Task A — Model 2: Multilayer Perceptron (MLP)

- The MLP is a fully-connected neural network applied to the flattened 2,187-dim pixel vector.
- Unlike logistic regression it learns non-linear combinations of pixels via hidden layers and ReLU activations, but like all flat-vector models it **discards spatial structure** — pixel adjacency is lost when the image is flattened.
- Both MLP and CNN are neural networks of similar capacity, so any performance gap isolates the value of the convolutional inductive bias. We train with the Adam optimiser and cross-entropy loss, recording training/validation loss and validation macro-F1 each epoch.

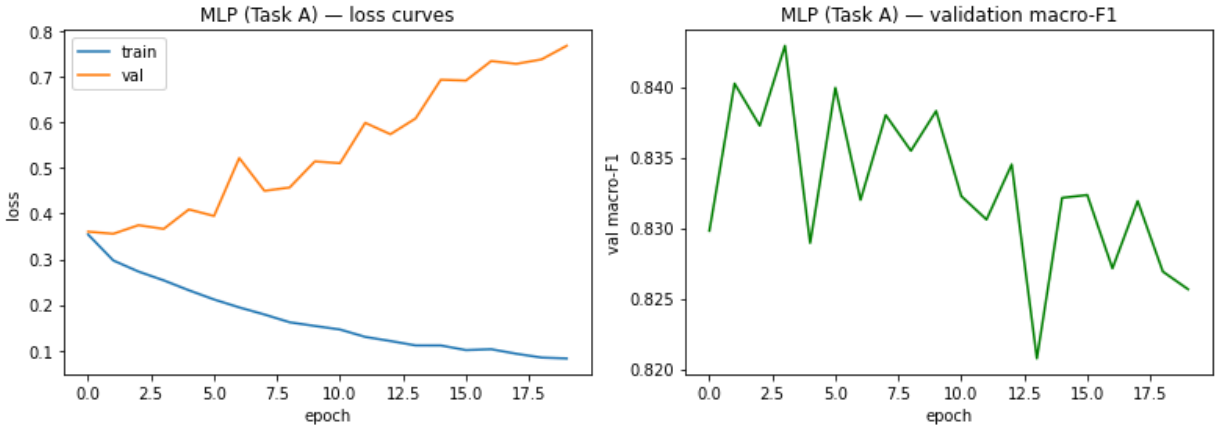
In [ ]:

```
# PyTorch setup + reusable training/eval utilities (shared by MLP and CNN)
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

torch.manual_seed(RANDOM_STATE)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Training device:", device)

def make_loader(X, y, batch_size=128, shuffle=False):
    """Wrap numpy arrays as a DataLoader. X may be flat (N,2187) for the MLP
    or image tensors (N,3,27,27) for the CNN."""
    X_t = torch.tensor(X, dtype=torch.float32)
    y_t = torch.tensor(y, dtype=torch.long)
    return DataLoader(TensorDataset(X_t, y_t), batch_size=batch_size, shuffle

def train_model(model, train_loader, val_loader, epochs=20, lr=1e-3,
                  class_weights=None, weight_decay=0.0, restore_best=False):
      """Train with Adam + cross-entropy; return per-epoch history.
      Options:
        class_weights : list/array of per-class weights for imbalanced data.
        weight_decay  : L2 regularisation strength (0.0 = off).
        restore_best  : if True, after training reload the weights from the e
                        with the highest validation macro-F1 (early stopping)
      model = model.to(device)
      w = (torch.tensor(class_weights, dtype=torch.float32).to(device)
           if class_weights is not None else None)
      criterion = nn.CrossEntropyLoss(weight=w)
      optimizer = torch.optim.Adam(model.parameters(), lr=lr,
                                   weight_decay=weight_decay)
      history = {"train_loss": [], "val_loss": [], "val_macro_f1": []}
      best_f1, best_state = -1.0, None
```

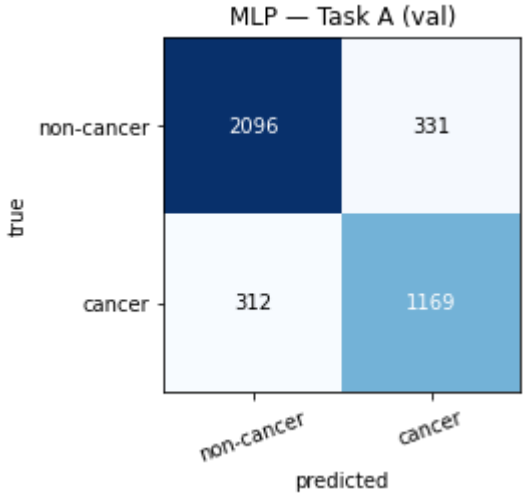
```
    for epoch in range(epochs):
        # ---- training ----
        model.train(); running = 0.0
        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad()
            loss = criterion(model(xb), yb)
            loss.backward(); optimizer.step()
            running += loss.item() * xb.size(0)
        train_loss = running / len(train_loader.dataset)

        # ---- validation ----
        model.eval(); vrun = 0.0; preds, trues = [], []
        with torch.no_grad():
            for xb, yb in val_loader:
                xb, yb = xb.to(device), yb.to(device)
                out = model(xb)
                vrun += criterion(out, yb).item() * xb.size(0)
                preds.append(out.argmax(1).cpu().numpy())
                trues.append(yb.cpu().numpy())
        val_loss = vrun / len(val_loader.dataset)
        vf1 = f1_score(np.concatenate(trues), np.concatenate(preds),
                       average="macro")

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["val_macro_f1"].append(vf1)
        if vf1 > best_f1:
            best_f1, best_state = vf1, copy.deepcopy(model.state_dict())
        print(f"  epoch {epoch+1:2d}: train_loss={train_loss:.4f} "
              f"val_loss={val_loss:.4f} val_macroF1={vf1:.3f}")

    if restore_best and best_state is not None:
        model.load_state_dict(best_state)
        print(f"  [early stopping: restored best epoch, val macro-F1={best_
    return history

def nn_predict(model, loader):
    model.eval(); preds = []
    with torch.no_grad():
        for xb, _ in loader:
            preds.append(model(xb.to(device)).argmax(1).cpu().numpy())
    return np.concatenate(preds)

def plot_history(history, title, save=None):
    fig, ax = plt.subplots(1, 2, figsize=(11, 4))
    ax[0].plot(history["train_loss"], label="train")
    ax[0].plot(history["val_loss"], label="val")
    ax[0].set_xlabel("epoch"); ax[0].set_ylabel("loss")
    ax[0].set_title(f"{title} — loss curves"); ax[0].legend()
    ax[1].plot(history["val_macro_f1"], color="green")
    ax[1].set_xlabel("epoch"); ax[1].set_ylabel("val macro-F1")
    ax[1].set_title(f"{title} — validation macro-F1")
    plt.tight_layout()
    plt.show()


# D.2: MLP on flattened, standardised pixels (Task A)
class MLP(nn.Module):
    def __init__(self, in_dim, n_classes, hidden=(256, 128), p=0.3):
        super().__init__()
        layers, d = [], in_dim
        for h in hidden:
            layers += [nn.Linear(d, h), nn.ReLU(), nn.Dropout(p)]
```

```
            d = h
        layers += [nn.Linear(d, n_classes)]
        self.net = nn.Sequential(*layers)
    def forward(self, x):
        return self.net(x)

# loaders use the flat standardised vectors from Step 7
trainA_loader = make_loader(XA_tr, yA_tr, batch_size=128, shuffle=True)
valA_loader   = make_loader(XA_va, yA_va, batch_size=256, shuffle=False)

mlpA = MLP(in_dim=XA_tr.shape[1], n_classes=2)
print("Training MLP (Task A)...")
histA_mlp = train_model(mlpA, trainA_loader, valA_loader, epochs=20, lr=1e-3)

plot_history(histA_mlp, "MLP (Task A)", save="loss_A_mlp")
evaluate("MLP", "A", yA_va, nn_predict(mlpA, valA_loader), A_names, save="cm_
```

```
Training device: cpu
Training MLP (Task A)...
  epoch  1: train_loss=0.3542 val_loss=0.3605 val_macroF1=0.830
  epoch  2: train_loss=0.2974 val_loss=0.3558 val_macroF1=0.840
  epoch  3: train_loss=0.2731 val_loss=0.3746 val_macroF1=0.837
  epoch  4: train_loss=0.2538 val_loss=0.3665 val_macroF1=0.843
  epoch  5: train_loss=0.2321 val_loss=0.4091 val_macroF1=0.829
  epoch  6: train_loss=0.2119 val_loss=0.3948 val_macroF1=0.840
  epoch  7: train_loss=0.1945 val_loss=0.5218 val_macroF1=0.832
  epoch  8: train_loss=0.1790 val_loss=0.4499 val_macroF1=0.838
  epoch  9: train_loss=0.1620 val_loss=0.4572 val_macroF1=0.835
  epoch 10: train_loss=0.1539 val_loss=0.5143 val_macroF1=0.838
  epoch 11: train_loss=0.1461 val_loss=0.5104 val_macroF1=0.832
  epoch 12: train_loss=0.1299 val_loss=0.5989 val_macroF1=0.831
  epoch 13: train_loss=0.1209 val_loss=0.5739 val_macroF1=0.835
  epoch 14: train_loss=0.1113 val_loss=0.6085 val_macroF1=0.821
  epoch 15: train_loss=0.1112 val_loss=0.6932 val_macroF1=0.832
  epoch 16: train_loss=0.1010 val_loss=0.6915 val_macroF1=0.832
  epoch 17: train_loss=0.1029 val_loss=0.7342 val_macroF1=0.827
  epoch 18: train_loss=0.0930 val_loss=0.7281 val_macroF1=0.832
  epoch 19: train_loss=0.0847 val_loss=0.7378 val_macroF1=0.827
  epoch 20: train_loss=0.0827 val_loss=0.7677 val_macroF1=0.826
[saved] plots/loss_A_mlp.png
```




```
[Task A] MLP (val): accuracy=0.835 | macro-F1=0.826
              precision    recall  f1-score   support

  non-cancer      0.870     0.864     0.867      2427
      cancer      0.779     0.789     0.784      1481

    accuracy                          0.835      3908
   macro avg      0.825     0.826     0.826      3908
weighted avg      0.836     0.835     0.836      3908

[saved] plots/cm_A_mlp.png
```



Out[ ]: 0.8256559728331136

- MLP — loss curves: classic overfitting. Train loss falls steadily toward 0.08 while validation loss bottoms out around epoch 1–2 (~0.36) then climbs to ~0.77. Validation macro-F1 peaks early (~0.843 at epoch 3) and drifts down to ~0.83, which indicates that the model is memorising training pixels, not learning generalisable structure.
- The MLP performs better both classical models, its non-linear hidden layers capture pixel interactions since the LogReg cannot, and unlike the tree it doesn't overfit catastrophically (dropout + gradient training regularise it).
- But it's still flat, it threw away pixel geometry, F1 is 0.83 below the target 0.90.

D.3 Task A — Model 3: Convolutional Neural Network (CNN) + hyperparameter tuning

- The CNN operates on the 3×27×27 image tensor and applies stacks of convolution + ReLU + max-pooling, so each filter responds to a *local* spatial neighbourhood and the same filter is shared across the image (weight sharing).
- This inductive bias is suited to nuclear morphology — the discriminative signal our EDA located in local texture rather than global pixel intensity.
- We tune the **learning rate** (1e-2, 1e-3, 1e-4) on the validation set, examining its effect on convergence and validation macro-F1, and select the best value for the final model.
- Input images are normalised with the per-channel statistics computed on the training split only.

In [18]:
```
# D.3a: build normalised image tensors (N, 3, 27, 27) for the CNN
def make_image_loader(X, idx, y, mean=CH_MEAN, std=CH_STD,
                      batch_size=128, shuffle=False):
    imgs = X[idx].astype(np.float32) / 255.0
    imgs = (imgs - mean) / std
    imgs = np.transpose(imgs, (0, 3, 1, 2))   # NHWC -> NCHW
    return make_loader(imgs, y[idx], batch_size, shuffle)

trainA_img = make_image_loader(XA, trainA_idx, yA, batch_size=128, shuffle=Tr
valA_img   = make_image_loader(XA, valA_idx,   yA, batch_size=256, shuffle=Fa
print("Image loaders ready.")
```

```
Image loaders ready.
```

In [19]:
```
# D.3b: a compact CNN for 27x27 RGB images
class SimpleCNN(nn.Module):
    def __init__(self, n_classes, p=0.3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),  nn.ReLU(), nn.MaxPool2d(2),  # 2
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 1
            nn.Conv2d(64, 128, 3, padding=1),nn.ReLU(), nn.MaxPool2d(2),  # 6
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(p),
            nn.Linear(128 * 3 * 3, 128), nn.ReLU(),
            nn.Dropout(p),
            nn.Linear(128, n_classes),
        )
    def forward(self, x):
        return self.classifier(self.features(x))
```

In [20]:
```
# D.3c: tune the learning rate on the validation set
candidate_lrs = [1e-2, 1e-3, 1e-4]
lr_runs = {}

for lr in candidate_lrs:
    torch.manual_seed(RANDOM_STATE)          # same init each run -> fair com
    model = SimpleCNN(n_classes=2)
    print(f"\n=== CNN training, lr={lr} ===")
    hist = train_model(model, trainA_img, valA_img, epochs=15, lr=lr)
    lr_runs[lr] = {"history": hist, "model": model}

# Summary: best validation macro-F1 reached by each learning rate
print("\nLearning-rate sweep (best val macro-F1):")
for lr in candidate_lrs:
    print(f"  lr={lr}: {max(lr_runs[lr]['history']['val_macro_f1']):.3f}")
```

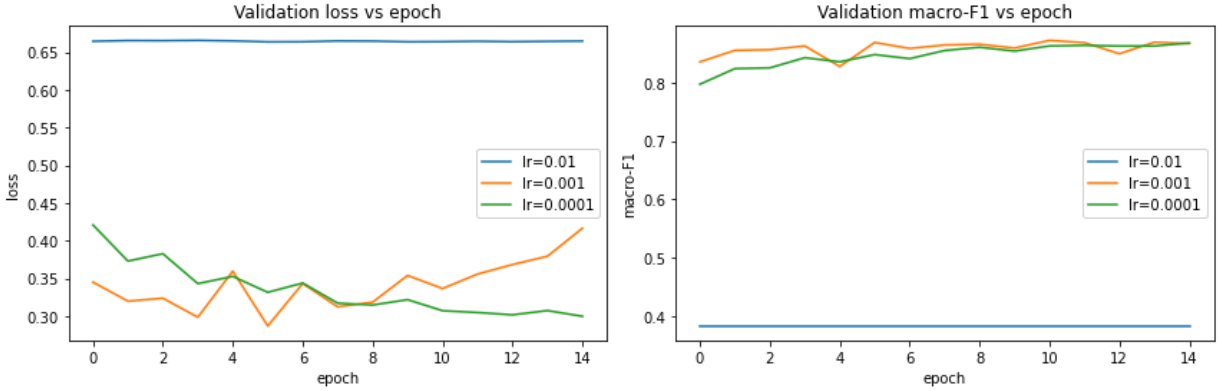
```
=== CNN training, lr=0.01 ===
  epoch  1: train_loss=0.8116 val_loss=0.6643 val_macroF1=0.383
  epoch  2: train_loss=0.6582 val_loss=0.6653 val_macroF1=0.383
  epoch  3: train_loss=0.6575 val_loss=0.6651 val_macroF1=0.383
  epoch  4: train_loss=0.6570 val_loss=0.6655 val_macroF1=0.383
  epoch  5: train_loss=0.6571 val_loss=0.6648 val_macroF1=0.383
  epoch  6: train_loss=0.6569 val_loss=0.6636 val_macroF1=0.383
  epoch  7: train_loss=0.6571 val_loss=0.6637 val_macroF1=0.383
  epoch  8: train_loss=0.6568 val_loss=0.6647 val_macroF1=0.383
  epoch  9: train_loss=0.6570 val_loss=0.6646 val_macroF1=0.383
  epoch 10: train_loss=0.6569 val_loss=0.6636 val_macroF1=0.383
  epoch 11: train_loss=0.6569 val_loss=0.6638 val_macroF1=0.383
  epoch 12: train_loss=0.6567 val_loss=0.6643 val_macroF1=0.383
  epoch 13: train_loss=0.6570 val_loss=0.6638 val_macroF1=0.383
  epoch 14: train_loss=0.6568 val_loss=0.6642 val_macroF1=0.383
  epoch 15: train_loss=0.6570 val_loss=0.6646 val_macroF1=0.383

=== CNN training, lr=0.001 ===
  epoch  1: train_loss=0.3645 val_loss=0.3450 val_macroF1=0.835
  epoch  2: train_loss=0.2927 val_loss=0.3197 val_macroF1=0.855
  epoch  3: train_loss=0.2704 val_loss=0.3237 val_macroF1=0.856
  epoch  4: train_loss=0.2439 val_loss=0.2985 val_macroF1=0.863
  epoch  5: train_loss=0.2305 val_loss=0.3596 val_macroF1=0.828
  epoch  6: train_loss=0.2208 val_loss=0.2868 val_macroF1=0.869
  epoch  7: train_loss=0.1988 val_loss=0.3431 val_macroF1=0.858
  epoch  8: train_loss=0.1900 val_loss=0.3122 val_macroF1=0.864
  epoch  9: train_loss=0.1637 val_loss=0.3184 val_macroF1=0.866
  epoch 10: train_loss=0.1477 val_loss=0.3538 val_macroF1=0.859
  epoch 11: train_loss=0.1364 val_loss=0.3365 val_macroF1=0.872
```

```
  epoch 12: train_loss=0.1113 val_loss=0.3556 val_macroF1=0.868
  epoch 13: train_loss=0.1000 val_loss=0.3683 val_macroF1=0.849
  epoch 14: train_loss=0.0894 val_loss=0.3793 val_macroF1=0.869
  epoch 15: train_loss=0.0824 val_loss=0.4164 val_macroF1=0.867

=== CNN training, lr=0.0001 ===
  epoch  1: train_loss=0.4911 val_loss=0.4208 val_macroF1=0.797
  epoch  2: train_loss=0.3387 val_loss=0.3729 val_macroF1=0.824
  epoch  3: train_loss=0.3145 val_loss=0.3827 val_macroF1=0.825
  epoch  4: train_loss=0.2991 val_loss=0.3430 val_macroF1=0.843
  epoch  5: train_loss=0.3009 val_loss=0.3525 val_macroF1=0.835
  epoch  6: train_loss=0.2829 val_loss=0.3315 val_macroF1=0.848
  epoch  7: train_loss=0.2798 val_loss=0.3437 val_macroF1=0.841
  epoch  8: train_loss=0.2715 val_loss=0.3171 val_macroF1=0.855
  epoch  9: train_loss=0.2619 val_loss=0.3147 val_macroF1=0.861
  epoch 10: train_loss=0.2664 val_loss=0.3217 val_macroF1=0.854
  epoch 11: train_loss=0.2562 val_loss=0.3071 val_macroF1=0.863
  epoch 12: train_loss=0.2494 val_loss=0.3047 val_macroF1=0.864
  epoch 13: train_loss=0.2444 val_loss=0.3016 val_macroF1=0.863
  epoch 14: train_loss=0.2418 val_loss=0.3073 val_macroF1=0.863
  epoch 15: train_loss=0.2343 val_loss=0.2997 val_macroF1=0.868

Learning-rate sweep (best val macro-F1):
  lr=0.01: 0.383
  lr=0.001: 0.872
  lr=0.0001: 0.868
```

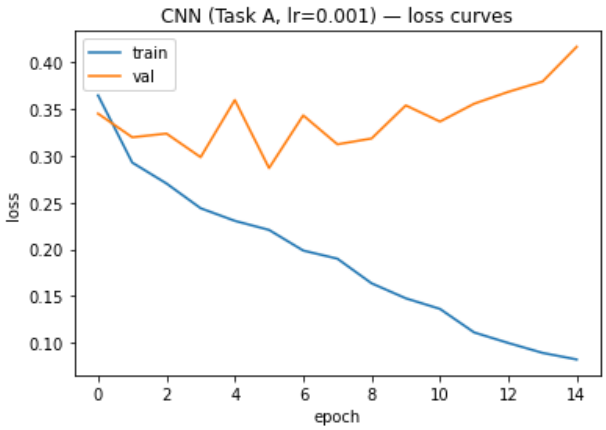
In [ ]:

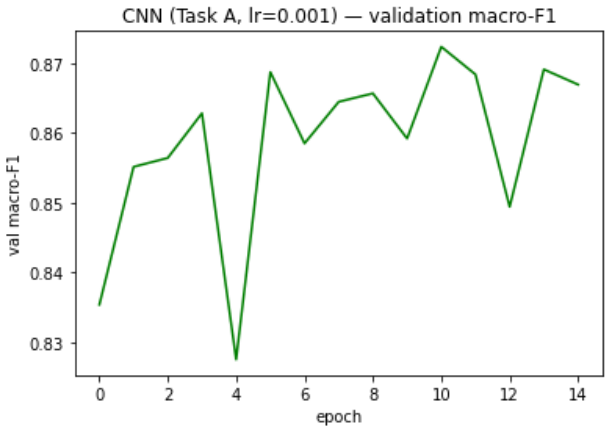
```
# D.3d: visualise tuning, select best lr, evaluate on validation
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
for lr in candidate_lrs:
    ax[0].plot(lr_runs[lr]["history"]["val_loss"], label=f"lr={lr}")
    ax[1].plot(lr_runs[lr]["history"]["val_macro_f1"], label=f"lr={lr}")
ax[0].set_title("Validation loss vs epoch"); ax[0].set_xlabel("epoch"); ax[0]
ax[1].set_title("Validation macro-F1 vs epoch"); ax[1].set_xlabel("epoch"); a
plt.tight_layout();
plt.show()

# Select the lr with the highest best-epoch validation macro-F1
best_lr = max(candidate_lrs,
              key=lambda lr: max(lr_runs[lr]["history"]["val_macro_f1"]))
print("Best learning rate:", best_lr)

cnnA = lr_runs[best_lr]["model"]
plot_history(lr_runs[best_lr]["history"], f"CNN (Task A, lr={best_lr})", save
evaluate("CNN", "A", yA_va, nn_predict(cnnA, valA_img), A_names, save="cm_A_c
```

```
[saved] plots/cnn_A_lr_tuning.png
```

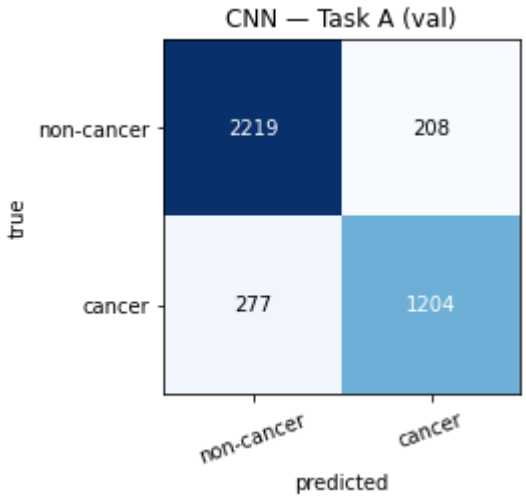



```
Best learning rate: 0.001
[saved] plots/loss_A_cnn.png
```






```
[Task A] CNN (val): accuracy=0.876 | macro-F1=0.867
              precision    recall  f1-score   support

  non-cancer      0.889     0.914     0.901      2427
      cancer      0.853     0.813     0.832      1481

    accuracy                          0.876      3908
   macro avg      0.871     0.864     0.867      3908
weighted avg      0.875     0.876     0.875      3908

[saved] plots/cm_A_cnn.png
```



Out[ ]: `0.8669183967492751`

D.3 observations (Task A CNN). :

- The learning-rate sweep confirmed lr=1e-3 as best (1e-2 unstable, 1e-4 underfits within 15 epochs).
- The macro-F1 of CNN is 0.867, clearly outperforming the previous models. It demonstrates that the convolutional inductive bias, which exploits local spatial structure. It becomes the key ingredient for this task.
- However, the loss curves show **overfitting**: training loss falls to ~0.08 while validation loss bottoms out around epoch 5 (~0.29) and then rises to ~0.42. It makes a widening generalisation gap.
- The CNN also shows lower recall on the minority *cancer* class (0.813 vs 0.914), indicating class imbalance as a second lever.

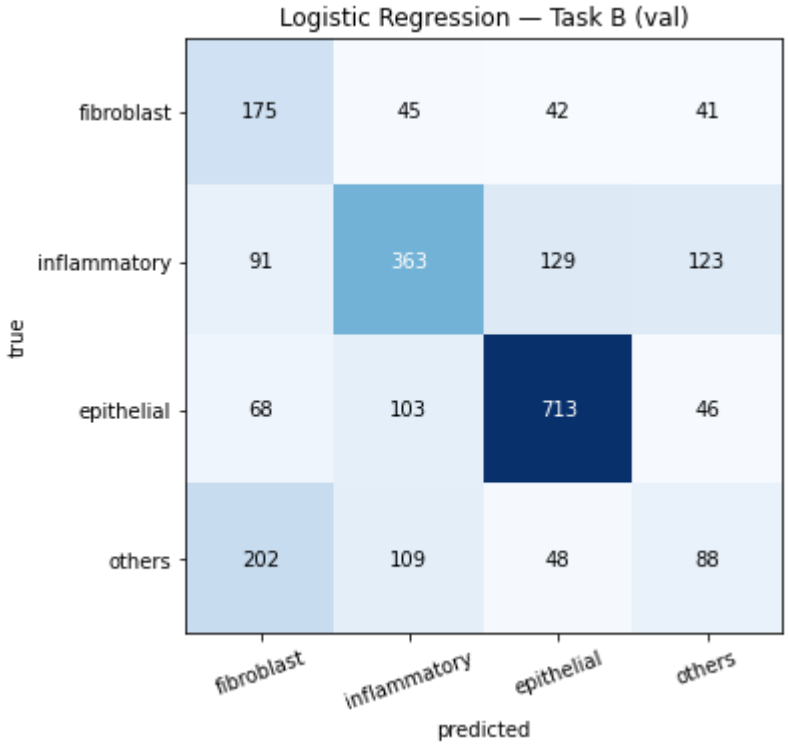
D.4 Task B — Models 1–3 (reuse pipeline, swap labels/output)

Task B reuses the identical pipeline — patient-level split, scaling, and the four model families — with two changes: the target is the 4-class `cellType`, and each model's output layer

has four units.

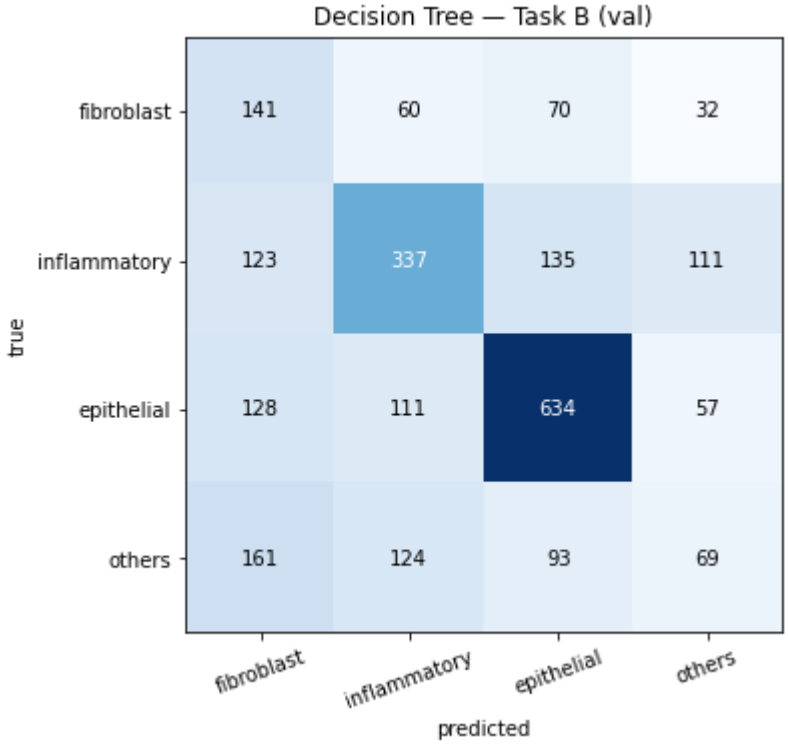
In [30]:
```
# D.4a: Task B classical baselines
B_names = [celltype_map[i] for i in range(4)]   # class names ordered by inde
print("Task B classes (index order):", B_names)

logregB = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE).fit(XB
evaluate("Logistic Regression", "B", yB_va, logregB.predict(XB_va),
         B_names, save="cm_B_logreg")

treeB = DecisionTreeClassifier(random_state=RANDOM_STATE).fit(XB_tr, yB_tr)
evaluate("Decision Tree", "B", yB_va, treeB.predict(XB_va),
         B_names, save="cm_B_tree")
```

```
Task B classes (index order): ['fibroblast', 'inflammatory', 'epithelial', 'o
thers']
/Users/hieutong/opt/anaconda3/lib/python3.8/site-packages/sklearn/linear_mode
l/_logistic.py:460: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres
sion
  n_iter_i = _check_optimize_result(
[Task B] Logistic Regression (val): accuracy=0.561 | macro-F1=0.492
              precision    recall  f1-score   support

  fibroblast      0.326     0.578     0.417       303
inflammatory      0.585     0.514     0.548       706
  epithelial      0.765     0.767     0.766       930
      others      0.295     0.197     0.236       447

    accuracy                          0.561      2386
   macro avg      0.493     0.514     0.492      2386
weighted avg      0.568     0.561     0.558      2386

[saved] plots/cm_B_logreg.png
```




```
[Task B] Decision Tree (val): accuracy=0.495 | macro-F1=0.427
              precision    recall  f1-score   support
```

```
  fibroblast      0.255     0.465     0.329       303
inflammatory      0.533     0.477     0.504       706
  epithelial      0.680     0.682     0.681       930
      others      0.257     0.154     0.193       447

    accuracy                          0.495      2386
   macro avg      0.431     0.445     0.427      2386
weighted avg      0.503     0.495     0.492      2386

[saved] plots/cm_B_tree.png
```



Out[30]: 0.42672544700722725

Observations:

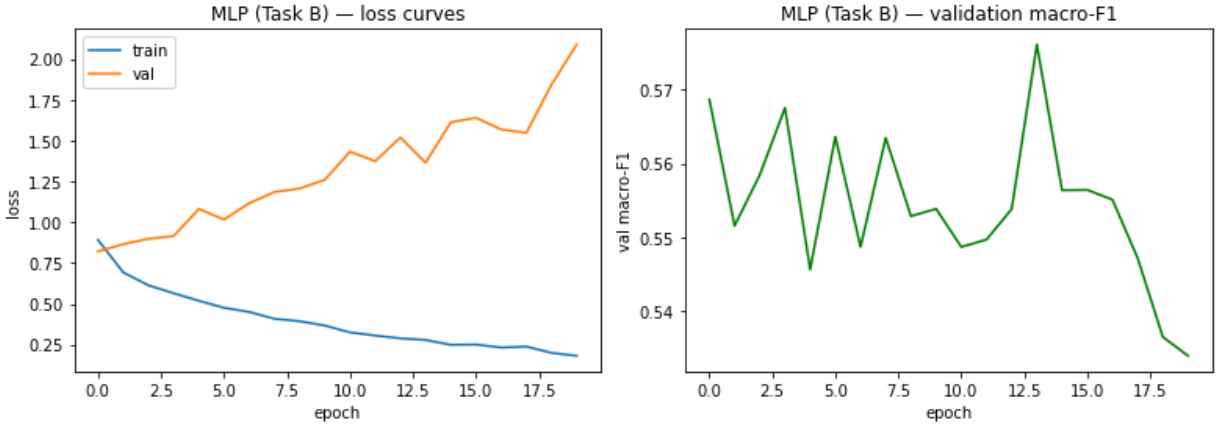
- LogReg 0.492 > Tree 0.427, which is the same pattern as Task A: the linear model beats the unconstrained tree, which overfits the 2,187 noisy pixel features (high variance).
- Both sit well under the 0.60 target. Macro-F1 is dragged down by the minority classes — others and fibroblast — which share visual appearance with other types, while epithelial (largest, most distinct) scores highest.
- Flat pixel models cannot separate these subtle morphological classes.

In [31]:

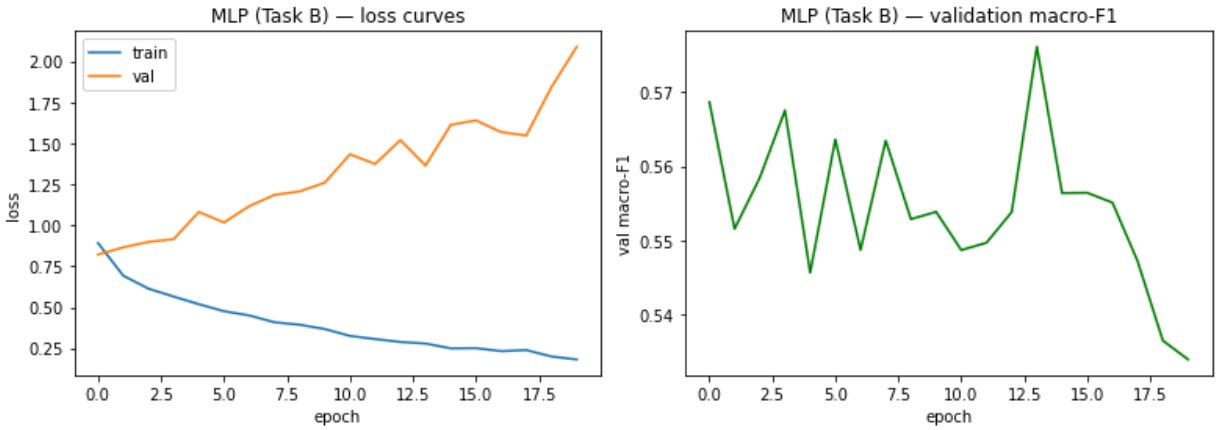
```
# D.4b: Task B MLP
trainB_loader = make_loader(XB_tr, yB_tr, batch_size=128, shuffle=True)
valB_loader   = make_loader(XB_va, yB_va, batch_size=256, shuffle=False)

torch.manual_seed(RANDOM_STATE)
mlpB = MLP(in_dim=XB_tr.shape[1], n_classes=4)
print("Training MLP (Task B)...")
histB_mlp = train_model(mlpB, trainB_loader, valB_loader, epochs=20, lr=1e-3)
plot_history(histB_mlp, "MLP (Task B)", save="loss_B_mlp")
trainB_loader = make_loader(XB_tr, yB_tr, batch_size=128, shuffle=True)
valB_loader   = make_loader(XB_va, yB_va, batch_size=256, shuffle=False)

torch.manual_seed(RANDOM_STATE)
mlpB = MLP(in_dim=XB_tr.shape[1], n_classes=4)
print("Training MLP (Task B)...")
histB_mlp = train_model(mlpB, trainB_loader, valB_loader, epochs=20, lr=1e-3)
```

```
plot_history(histB_mlp, "MLP (Task B)", save="loss_B_mlp")
evaluate("MLP", "B", yB_va, nn_predict(mlpB, valB_loader), B_names, save="cm_
```

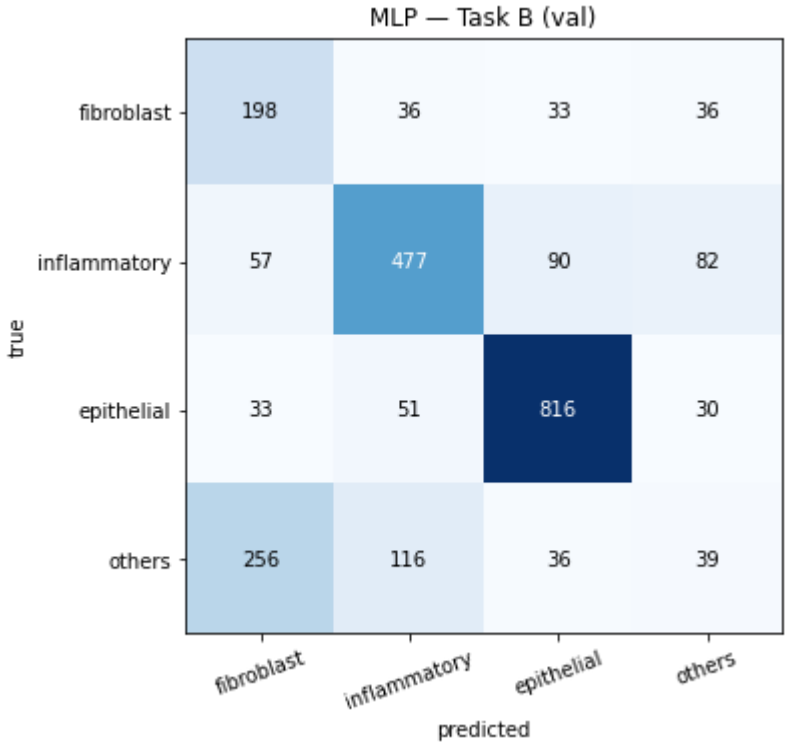
```
Training MLP (Task B)...
  epoch  1: train_loss=0.8913 val_loss=0.8213 val_macroF1=0.569
  epoch  2: train_loss=0.6927 val_loss=0.8655 val_macroF1=0.552
  epoch  3: train_loss=0.6135 val_loss=0.8984 val_macroF1=0.558
  epoch  4: train_loss=0.5652 val_loss=0.9154 val_macroF1=0.568
  epoch  5: train_loss=0.5188 val_loss=1.0814 val_macroF1=0.546
  epoch  6: train_loss=0.4757 val_loss=1.0171 val_macroF1=0.564
  epoch  7: train_loss=0.4501 val_loss=1.1167 val_macroF1=0.549
  epoch  8: train_loss=0.4085 val_loss=1.1856 val_macroF1=0.563
  epoch  9: train_loss=0.3935 val_loss=1.2066 val_macroF1=0.553
  epoch 10: train_loss=0.3670 val_loss=1.2601 val_macroF1=0.554
  epoch 11: train_loss=0.3251 val_loss=1.4329 val_macroF1=0.549
  epoch 12: train_loss=0.3059 val_loss=1.3740 val_macroF1=0.550
  epoch 13: train_loss=0.2883 val_loss=1.5197 val_macroF1=0.554
  epoch 14: train_loss=0.2787 val_loss=1.3646 val_macroF1=0.576
  epoch 15: train_loss=0.2492 val_loss=1.6128 val_macroF1=0.556
  epoch 16: train_loss=0.2506 val_loss=1.6401 val_macroF1=0.556
  epoch 17: train_loss=0.2323 val_loss=1.5686 val_macroF1=0.555
  epoch 18: train_loss=0.2385 val_loss=1.5476 val_macroF1=0.547
  epoch 19: train_loss=0.1997 val_loss=1.8436 val_macroF1=0.536
  epoch 20: train_loss=0.1816 val_loss=2.0892 val_macroF1=0.534
[saved] plots/loss_B_mlp.png
```




```
Training MLP (Task B)...
  epoch  1: train_loss=0.8913 val_loss=0.8213 val_macroF1=0.569
  epoch  2: train_loss=0.6927 val_loss=0.8655 val_macroF1=0.552
  epoch  3: train_loss=0.6135 val_loss=0.8984 val_macroF1=0.558
  epoch  4: train_loss=0.5652 val_loss=0.9154 val_macroF1=0.568
  epoch  5: train_loss=0.5188 val_loss=1.0814 val_macroF1=0.546
  epoch  6: train_loss=0.4757 val_loss=1.0171 val_macroF1=0.564
  epoch  7: train_loss=0.4501 val_loss=1.1167 val_macroF1=0.549
  epoch  8: train_loss=0.4085 val_loss=1.1856 val_macroF1=0.563
  epoch  9: train_loss=0.3935 val_loss=1.2066 val_macroF1=0.553
  epoch 10: train_loss=0.3670 val_loss=1.2601 val_macroF1=0.554
  epoch 11: train_loss=0.3251 val_loss=1.4329 val_macroF1=0.549
  epoch 12: train_loss=0.3059 val_loss=1.3740 val_macroF1=0.550
  epoch 13: train_loss=0.2883 val_loss=1.5197 val_macroF1=0.554
  epoch 14: train_loss=0.2787 val_loss=1.3646 val_macroF1=0.576
  epoch 15: train_loss=0.2492 val_loss=1.6128 val_macroF1=0.556
  epoch 16: train_loss=0.2506 val_loss=1.6401 val_macroF1=0.556
  epoch 17: train_loss=0.2323 val_loss=1.5686 val_macroF1=0.555
  epoch 18: train_loss=0.2385 val_loss=1.5476 val_macroF1=0.547
  epoch 19: train_loss=0.1997 val_loss=1.8436 val_macroF1=0.536
  epoch 20: train_loss=0.1816 val_loss=2.0892 val_macroF1=0.534
[saved] plots/loss_B_mlp.png
```




```
[Task B] MLP (val): accuracy=0.641 | macro-F1=0.534
              precision    recall  f1-score   support

  fibroblast      0.364     0.653     0.468       303
inflammatory      0.701     0.676     0.688       706
  epithelial      0.837     0.877     0.857       930
      others      0.209     0.087     0.123       447

    accuracy                          0.641      2386
   macro avg      0.528     0.573     0.534      2386
weighted avg      0.619     0.641     0.620      2386

[saved] plots/cm_B_mlp.png
```



Out[31]: `0.5338913650992938`

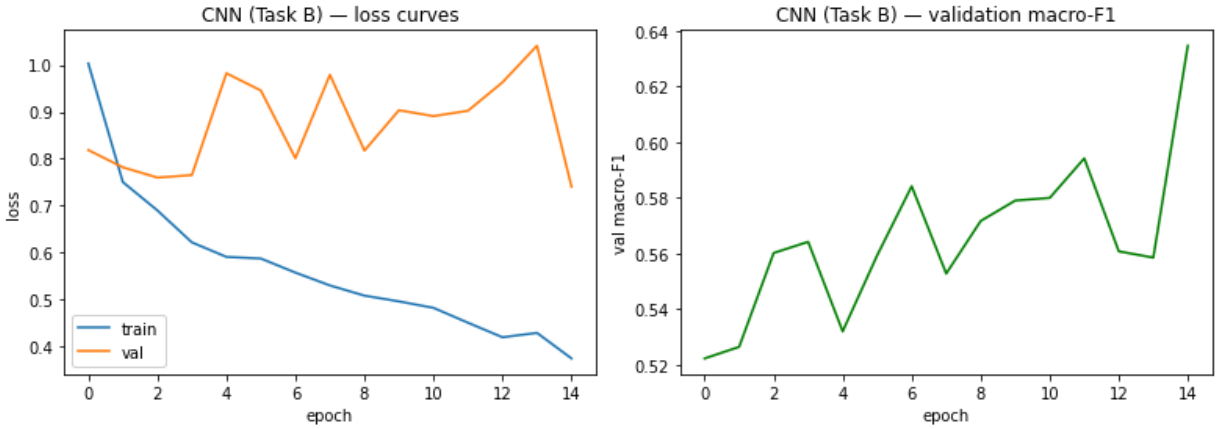
- Macro-F1 0.534, best of the flat models but well under 0.60.
- The pattern is the same as Task A but worse: train loss falls to 0.18 while val loss climbs from 0.82 to 2.09 — heavy overfitting from epoch 2. Note val macro-F1 actually peaked at 0.576 (epoch 14), so early stopping (restore_best) recovers ~0.04 for free — worth turning on.
- Per-class is the real story: epithelial strong (F1 0.857), but others collapses (F1 0.123, recall 0.087) — the model barely predicts it, and fibroblast leaks into it (precision 0.364).

Macro-F1 weights all four equally, so the minority failure dominates. This is the class-imbalance lever

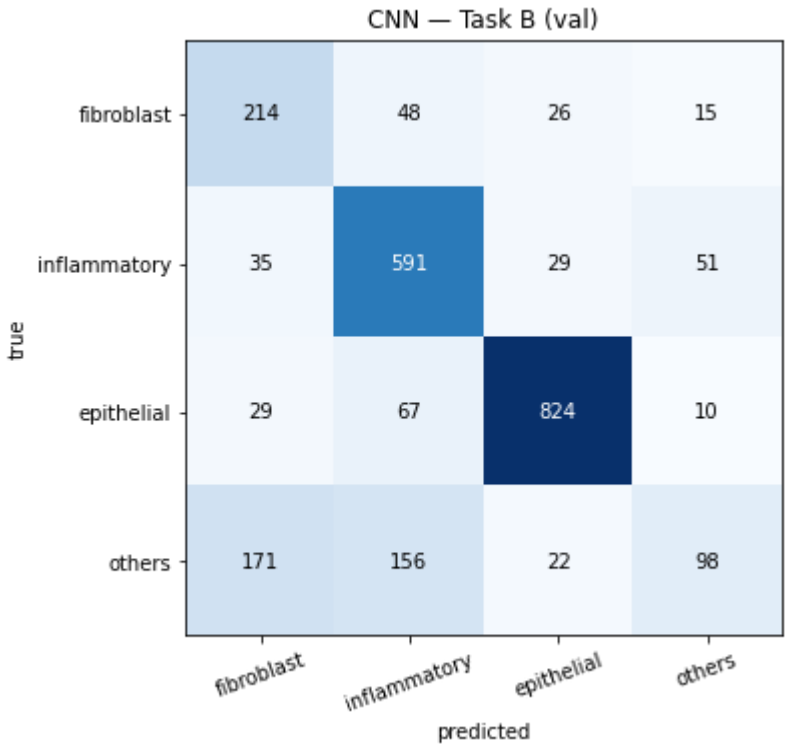
In [32]:

```
# D.4c: Task B CNN (own channel stats; reuses the best lr found on Task A)
trB = XB[trainB_idx].astype(np.float32) / 255.0
CH_MEAN_B = trB.mean(axis=(0, 1, 2))
CH_STD_B  = trB.std(axis=(0, 1, 2))

trainB_img = make_image_loader(XB, trainB_idx, yB, CH_MEAN_B, CH_STD_B,
                               batch_size=128, shuffle=True)
valB_img   = make_image_loader(XB, valB_idx, yB, CH_MEAN_B, CH_STD_B,
                               batch_size=256, shuffle=False)

torch.manual_seed(RANDOM_STATE)
cnnB = SimpleCNN(n_classes=4)
print(f"Training CNN (Task B, lr={best_lr})...")
histB_cnn = train_model(cnnB, trainB_img, valB_img, epochs=15, lr=best_lr)
plot_history(histB_cnn, "CNN (Task B)", save="loss_B_cnn")
evaluate("CNN", "B", yB_va, nn_predict(cnnB, valB_img), B_names, save="cm_B_c
```

```
Training CNN (Task B, lr=0.001)...
  epoch  1: train_loss=1.0032 val_loss=0.8184 val_macroF1=0.522
  epoch  2: train_loss=0.7500 val_loss=0.7816 val_macroF1=0.526
  epoch  3: train_loss=0.6896 val_loss=0.7597 val_macroF1=0.560
  epoch  4: train_loss=0.6213 val_loss=0.7649 val_macroF1=0.564
  epoch  5: train_loss=0.5903 val_loss=0.9824 val_macroF1=0.532
  epoch  6: train_loss=0.5868 val_loss=0.9454 val_macroF1=0.559
  epoch  7: train_loss=0.5567 val_loss=0.8010 val_macroF1=0.584
  epoch  8: train_loss=0.5293 val_loss=0.9790 val_macroF1=0.553
  epoch  9: train_loss=0.5076 val_loss=0.8172 val_macroF1=0.572
  epoch 10: train_loss=0.4953 val_loss=0.9034 val_macroF1=0.579
  epoch 11: train_loss=0.4815 val_loss=0.8908 val_macroF1=0.580
  epoch 12: train_loss=0.4498 val_loss=0.9023 val_macroF1=0.594
  epoch 13: train_loss=0.4187 val_loss=0.9630 val_macroF1=0.561
  epoch 14: train_loss=0.4281 val_loss=1.0410 val_macroF1=0.559
  epoch 15: train_loss=0.3734 val_loss=0.7400 val_macroF1=0.635
[saved] plots/loss_B_cnn.png
```




```
[Task B] CNN (val): accuracy=0.724 | macro-F1=0.635
              precision    recall  f1-score   support

  fibroblast      0.477     0.706     0.569       303
inflammatory      0.686     0.837     0.754       706
  epithelial      0.915     0.886     0.900       930
      others      0.563     0.219     0.316       447

    accuracy                          0.724      2386
   macro avg      0.660     0.662     0.635      2386
weighted avg      0.725     0.724     0.705      2386

[saved] plots/cm_B_cnn.png
```



Out[32]: 0.6346625123852097

- Epochs get macro-F1 0.635d which is greater than the 0.60 target and performancing better every flat model (MLP 0.534).
- The curve is noisy when val macro-F1 swings between 0.52 and 0.63 while train loss falls steadily.
- The best epoch (15, F1 0.635, val loss 0.74) happens to be the last.
- Train loss (0.37) still sits above val-loss noise, so it's less overfit than the MLP because the CNN's weight-sharing regularises.
- The spatial features lift every class versus the MLP, but the spread persists.
- Epithelial strongest (F1 0.900 — largest, most distinct). others weakest (F1 0.316): precision 0.563 but recall only 0.219. Fibroblast also leaks (precision 0.477). Macro-F1 weights all four equally, so others caps the score.

E. Advanced Techniques

We apply two advanced techniques.

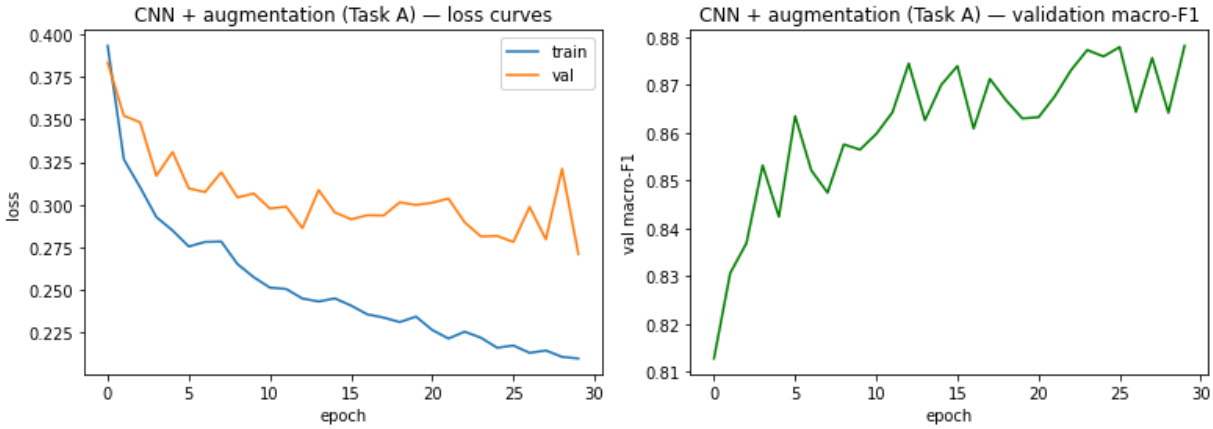
- **(E.1) Data augmentation** addresses the overfitting diagnosed in the Task A CNN: label-preserving random transformations of the training images enlarge the effective training set and discourage the model from memorising specific images. That reduces variance and improvs generalisation.
- **(E.2) Ensembling (soft voting)** combines several diverse models by averaging their predicted class probabilities; because the members make partly independent errors, averaging cancels noise and further reduces variance. Both build on the best-validation early stopping and L2 weight decay added to the training routine as additional regularisation.

In [33]:
```
# E.1: data augmentation for the Task A CNN
import copy
```

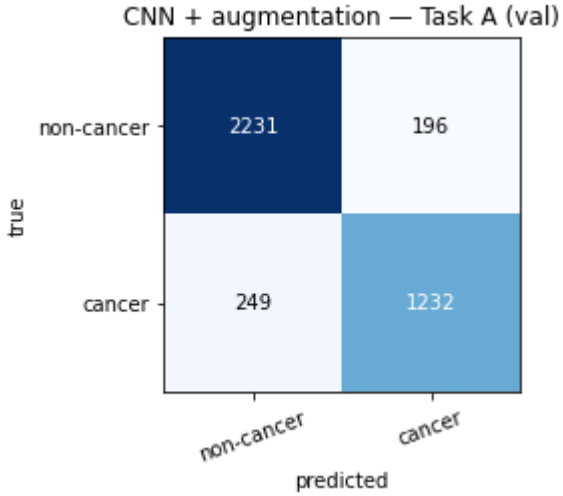
```
import torchvision.transforms as T
from torch.utils.data import Dataset

class AugmentedImageDataset(Dataset):
    """Applies label-preserving augmentation on the fly (train=True) or just
    normalisation (train=False). Input X is uint8 NHWC (0-255)."""
    def __init__(self, X, idx, y, mean, std, train=True):
        self.imgs = X[idx]
        self.y = y[idx]
        norm = T.Normalize(mean.tolist(), std.tolist())
        if train:
            self.tf = T.Compose([
                T.ToPILImage(),
                T.RandomHorizontalFlip(),
                T.RandomVerticalFlip(),
                T.RandomRotation(20),
                T.ColorJitter(brightness=0.1, contrast=0.1),
                T.ToTensor(),          # uint8 [0,255] -> float [0,1], HWC->C
                norm,
            ])
        else:
            self.tf = T.Compose([T.ToPILImage(), T.ToTensor(), norm])
    def __len__(self):
        return len(self.y)
    def __getitem__(self, i):
        return self.tf(self.imgs[i]), int(self.y[i])

trainA_aug = DataLoader(
    AugmentedImageDataset(XA, trainA_idx, yA, CH_MEAN, CH_STD, train=True),
    batch_size=128, shuffle=True)
valA_plain = DataLoader(
    AugmentedImageDataset(XA, valA_idx, yA, CH_MEAN, CH_STD, train=False),
    batch_size=256, shuffle=False)

torch.manual_seed(RANDOM_STATE)
cnnA_aug = SimpleCNN(n_classes=2)
print("Training augmented CNN (Task A)...")
histA_aug = train_model(cnnA_aug, trainA_aug, valA_plain,
                        epochs=30, lr=1e-3, weight_decay=1e-4, restore_best=T

plot_history(histA_aug, "CNN + augmentation (Task A)", save="loss_A_cnn_aug")
evaluate("CNN + augmentation", "A", yA_va, nn_predict(cnnA_aug, valA_plain),
         A_names, save="cm_A_cnn_aug")

# --- Task B: same augmentation, own channel stats (helps the weak 'others' c
trainB_aug = DataLoader(
    AugmentedImageDataset(XB, trainB_idx, yB, CH_MEAN_B, CH_STD_B, train=True
    batch_size=128, shuffle=True)
valB_plain = DataLoader(
    AugmentedImageDataset(XB, valB_idx, yB, CH_MEAN_B, CH_STD_B, train=False)
    batch_size=256, shuffle=False)

torch.manual_seed(RANDOM_STATE)
cnnB_aug = SimpleCNN(n_classes=4)
print("Training augmented CNN (Task B)...")
histB_aug = train_model(cnnB_aug, trainB_aug, valB_plain,
                        epochs=30, lr=best_lr, weight_decay=1e-4, restore_bes
plot_history(histB_aug, "CNN + augmentation (Task B)", save="loss_B_cnn_aug")
evaluate("CNN + augmentation", "B", yB_va, nn_predict(cnnB_aug, valB_plain),
          B_names, save="cm_B_cnn_aug")
```

```
Training augmented CNN (Task A)...
  epoch  1: train_loss=0.3933 val_loss=0.3834 val_macroF1=0.813
  epoch  2: train_loss=0.3266 val_loss=0.3522 val_macroF1=0.831
```

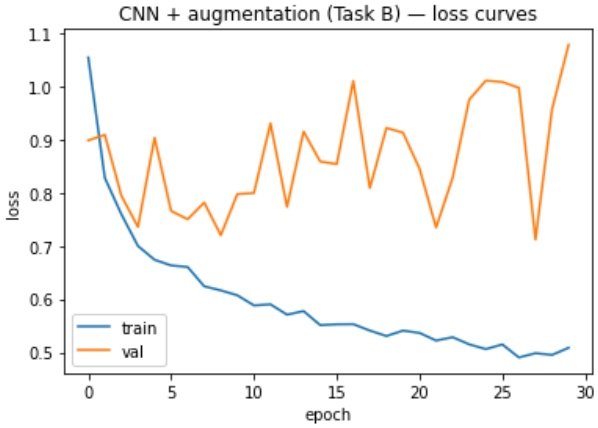
```
  epoch  3: train_loss=0.3102 val_loss=0.3483 val_macroF1=0.837
  epoch  4: train_loss=0.2927 val_loss=0.3170 val_macroF1=0.853
  epoch  5: train_loss=0.2848 val_loss=0.3310 val_macroF1=0.842
  epoch  6: train_loss=0.2754 val_loss=0.3095 val_macroF1=0.863
  epoch  7: train_loss=0.2781 val_loss=0.3074 val_macroF1=0.852
  epoch  8: train_loss=0.2783 val_loss=0.3190 val_macroF1=0.847
  epoch  9: train_loss=0.2651 val_loss=0.3042 val_macroF1=0.858
  epoch 10: train_loss=0.2574 val_loss=0.3066 val_macroF1=0.856
  epoch 11: train_loss=0.2512 val_loss=0.2977 val_macroF1=0.860
  epoch 12: train_loss=0.2504 val_loss=0.2988 val_macroF1=0.864
  epoch 13: train_loss=0.2448 val_loss=0.2862 val_macroF1=0.874
  epoch 14: train_loss=0.2431 val_loss=0.3086 val_macroF1=0.863
  epoch 15: train_loss=0.2449 val_loss=0.2954 val_macroF1=0.870
  epoch 16: train_loss=0.2407 val_loss=0.2914 val_macroF1=0.874
  epoch 17: train_loss=0.2355 val_loss=0.2938 val_macroF1=0.861
  epoch 18: train_loss=0.2337 val_loss=0.2936 val_macroF1=0.871
  epoch 19: train_loss=0.2310 val_loss=0.3013 val_macroF1=0.867
  epoch 20: train_loss=0.2342 val_loss=0.2998 val_macroF1=0.863
  epoch 21: train_loss=0.2264 val_loss=0.3011 val_macroF1=0.863
  epoch 22: train_loss=0.2213 val_loss=0.3035 val_macroF1=0.868
  epoch 23: train_loss=0.2254 val_loss=0.2896 val_macroF1=0.873
  epoch 24: train_loss=0.2218 val_loss=0.2813 val_macroF1=0.877
  epoch 25: train_loss=0.2159 val_loss=0.2816 val_macroF1=0.876
  epoch 26: train_loss=0.2172 val_loss=0.2780 val_macroF1=0.878
  epoch 27: train_loss=0.2129 val_loss=0.2986 val_macroF1=0.864
  epoch 28: train_loss=0.2143 val_loss=0.2796 val_macroF1=0.876
  epoch 29: train_loss=0.2105 val_loss=0.3210 val_macroF1=0.864
  epoch 30: train_loss=0.2096 val_loss=0.2710 val_macroF1=0.878
  [early stopping: restored best epoch, val macro-F1=0.878]
[saved] plots/loss_A_cnn_aug.png
```

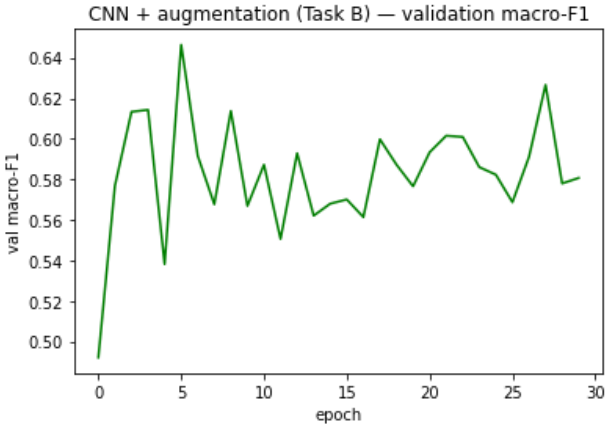



```
[Task A] CNN + augmentation (val): accuracy=0.886 | macro-F1=0.878
              precision    recall  f1-score   support

  non-cancer      0.900     0.919     0.909      2427
      cancer      0.863     0.832     0.847      1481

    accuracy                          0.886      3908
   macro avg      0.881     0.876     0.878      3908
weighted avg      0.886     0.886     0.886      3908

[saved] plots/cm_A_cnn_aug.png
```




```
Training augmented CNN (Task B)...
  epoch  1: train_loss=1.0540 val_loss=0.8984 val_macroF1=0.492
  epoch  2: train_loss=0.8279 val_loss=0.9086 val_macroF1=0.577
  epoch  3: train_loss=0.7595 val_loss=0.7943 val_macroF1=0.613
  epoch  4: train_loss=0.7000 val_loss=0.7357 val_macroF1=0.614
  epoch  5: train_loss=0.6741 val_loss=0.9032 val_macroF1=0.538
  epoch  6: train_loss=0.6635 val_loss=0.7658 val_macroF1=0.646
  epoch  7: train_loss=0.6605 val_loss=0.7501 val_macroF1=0.592
  epoch  8: train_loss=0.6242 val_loss=0.7815 val_macroF1=0.568
  epoch  9: train_loss=0.6165 val_loss=0.7202 val_macroF1=0.614
  epoch 10: train_loss=0.6073 val_loss=0.7974 val_macroF1=0.567
  epoch 11: train_loss=0.5883 val_loss=0.7993 val_macroF1=0.587
  epoch 12: train_loss=0.5903 val_loss=0.9303 val_macroF1=0.551
  epoch 13: train_loss=0.5709 val_loss=0.7733 val_macroF1=0.593
  epoch 14: train_loss=0.5776 val_loss=0.9148 val_macroF1=0.562
  epoch 15: train_loss=0.5513 val_loss=0.8587 val_macroF1=0.568
  epoch 16: train_loss=0.5528 val_loss=0.8537 val_macroF1=0.570
  epoch 17: train_loss=0.5531 val_loss=1.0101 val_macroF1=0.561
  epoch 18: train_loss=0.5410 val_loss=0.8089 val_macroF1=0.600
  epoch 19: train_loss=0.5308 val_loss=0.9217 val_macroF1=0.587
  epoch 20: train_loss=0.5409 val_loss=0.9128 val_macroF1=0.577
  epoch 21: train_loss=0.5367 val_loss=0.8453 val_macroF1=0.593
  epoch 22: train_loss=0.5223 val_loss=0.7346 val_macroF1=0.602
  epoch 23: train_loss=0.5286 val_loss=0.8283 val_macroF1=0.601
  epoch 24: train_loss=0.5151 val_loss=0.9748 val_macroF1=0.586
  epoch 25: train_loss=0.5062 val_loss=1.0106 val_macroF1=0.582
  epoch 26: train_loss=0.5150 val_loss=1.0081 val_macroF1=0.569
  epoch 27: train_loss=0.4905 val_loss=0.9971 val_macroF1=0.591
  epoch 28: train_loss=0.4985 val_loss=0.7121 val_macroF1=0.627
  epoch 29: train_loss=0.4954 val_loss=0.9569 val_macroF1=0.578
  epoch 30: train_loss=0.5086 val_loss=1.0780 val_macroF1=0.581
  [early stopping: restored best epoch, val macro-F1=0.646]
[saved] plots/loss_B_cnn_aug.png
```

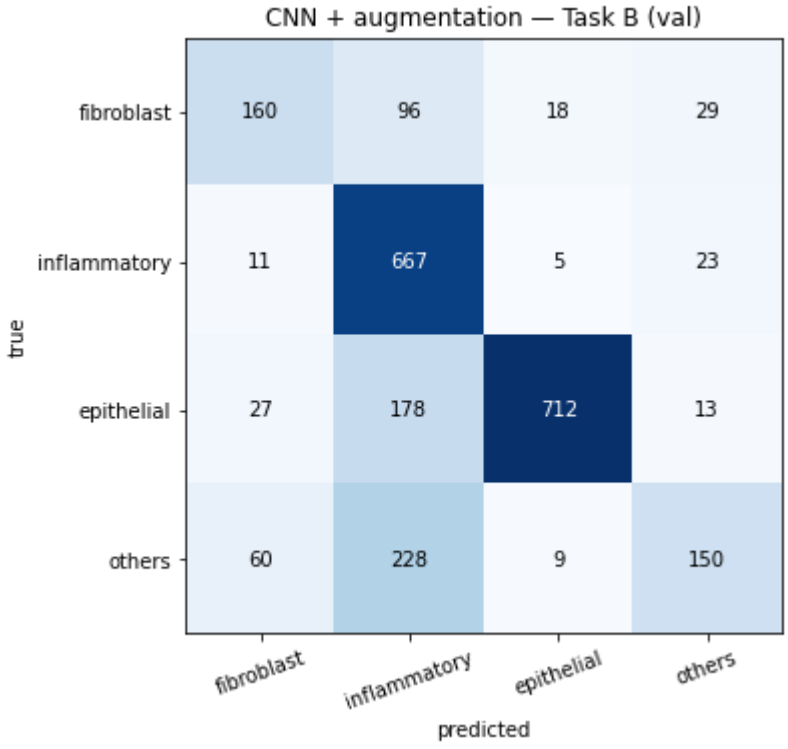





```
[Task B] CNN + augmentation (val): accuracy=0.708 | macro-F1=0.646
              precision    recall  f1-score   support

  fibroblast      0.620     0.528     0.570       303
```

```
inflammatory      0.571     0.945     0.711       706
  epithelial      0.957     0.766     0.851       930
      others      0.698     0.336     0.453       447

    accuracy                          0.708      2386
   macro avg      0.711     0.643     0.646      2386
weighted avg      0.751     0.708     0.699      2386
```

```
[saved] plots/cm_B_cnn_aug.png
```



Out[33]: `0.6464264907502615`

CNN + augmentation — Task A:

- Val macro-F1 increases to 0.878 from 0.867.
- Augmentation cures the overfitting the plain CNN showed: across all 30 epochs train and val loss fall together (train 0.39→0.21, val 0.38→0.27) instead of diverging, and val macro-F1 climbs smoothly to its peak at the final epoch. Flips and rotations are label-preserving for cells (no canonical orientation), so they add free variety and regularise.
- The confusion matrix shows balanced, strong classes — non-cancer F1 0.909, cancer 0.847, but cancer recall (0.832) is the ceiling keeping it just under 0.90: a minority of cancer cells still get called non-cancer.

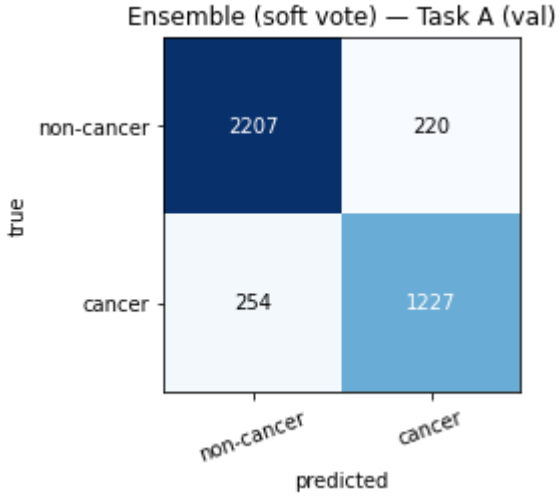
CNN + augmentation — Task :

- Val macro-F1 increases to 0.646 from 0.635.
- Training is noisy here: val macro-F1 swings 0.49–0.65 while train loss falls steadily, and the best epoch (6) comes early and is never beaten, so restore_best matters — the last epoch scored only 0.581.
- The volatility comes from the 4-class minority structure. It shows that augmentation helped the weak class most (others recall 0.219→0.336, F1 0.316→0.453), but inflammatory now over-fires (recall 0.945, precision 0.571) and epithelial recall dropped (0.886→0.766).
- Gains on rare classes are cancelled by this new confusion, so macro-F1 barely moves. Augmentation redistributes errors rather than removing them.

Currently, Augmented CNN is the best model on both tasks.

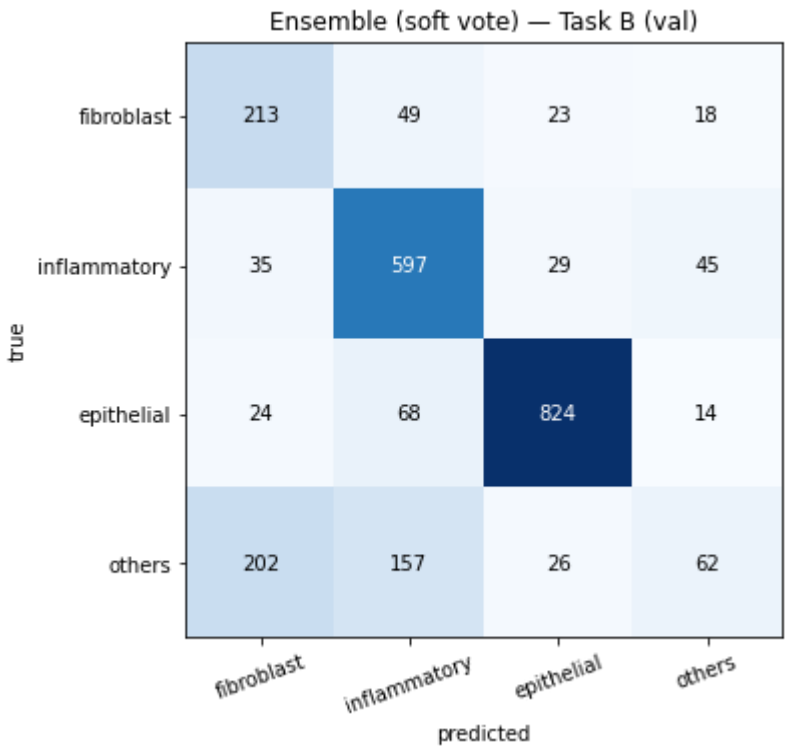
In [34]:
```
# E.2: ensemble by soft voting (averaging predicted probabilities).
# Members are deliberately DIVERSE: an MLP (global pixel interactions), the t
# (local spatial features), and the augmentation-regularised CNN. Averaging t
# probabilities of accurate-but-decorrelated models cancels their independent
# reducing variance without adding bias. No retraining needed (reuses D + E.1

def nn_proba(model, loader):
    """Softmax class probabilities in loader order (val loaders are shuffle=F
    so every member returns rows aligned to the same validation samples)."""
    model.eval(); out = []
    with torch.no_grad():
        for xb, _ in loader:
            out.append(torch.softmax(model(xb.to(device)), dim=1).cpu().numpy
    return np.concatenate(out)

def soft_vote(members):
    """members: list of (model, loader). Returns averaged-probability predict
    probs = np.mean([nn_proba(m, l) for m, l in members], axis=0)
    return probs.argmax(1)

# --- Task A ensemble: MLP + tuned CNN + augmented CNN ---
membersA = [(mlpA, valA_loader), (cnnA, valA_img), (cnnA_aug, valA_plain)]
predA_ens = soft_vote(membersA)
evaluate("Ensemble (soft vote)", "A", yA_va, predA_ens, A_names, save="cm_A_e

# --- Task B ensemble: MLP + CNN + augmented CNN ---
membersB = [(mlpB, valB_loader), (cnnB, valB_img), (cnnB_aug, valB_plain)]
predB_ens = soft_vote(membersB)
evaluate("Ensemble (soft vote)", "B", yB_va, predB_ens, B_names, save="cm_B_e
```

```
[Task A] Ensemble (soft vote) (val): accuracy=0.879 | macro-F1=0.871
              precision    recall  f1-score   support

  non-cancer      0.897     0.909     0.903      2427
      cancer      0.848     0.828     0.838      1481

    accuracy                          0.879      3908
   macro avg      0.872     0.869     0.871      3908
weighted avg      0.878     0.879     0.878      3908

[saved] plots/cm_A_ensemble.png
```




```
[Task B] Ensemble (soft vote) (val): accuracy=0.711 | macro-F1=0.604
              precision    recall  f1-score   support

   fibroblast      0.449     0.703     0.548       303
 inflammatory      0.685     0.846     0.757       706
```

```
  epithelial      0.914     0.886     0.900       930
      others      0.446     0.139     0.212       447

    accuracy                          0.711      2386
   macro avg      0.624     0.643     0.604      2386
weighted avg      0.700     0.711     0.684      2386

[saved] plots/cm_B_ensemble.png
```



Out[34]: 0.6041409402385702

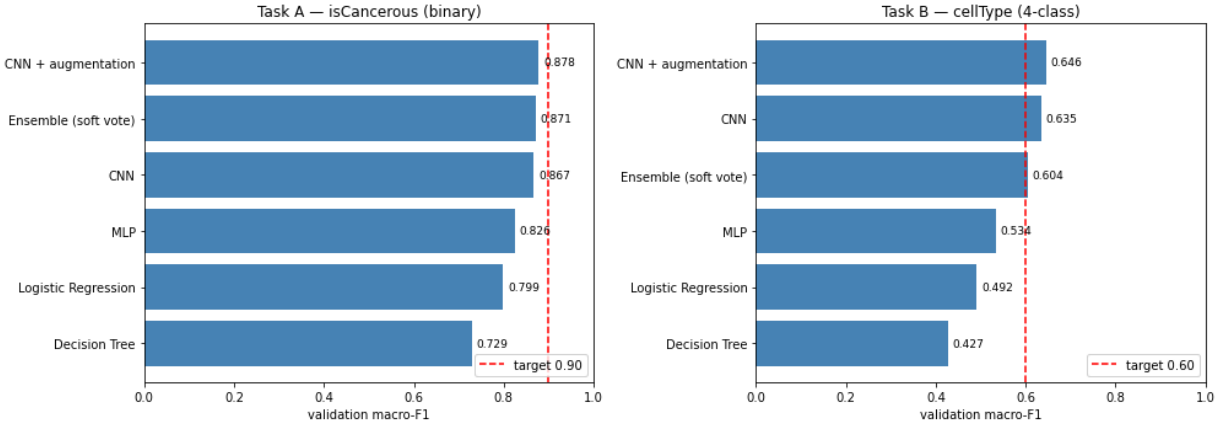
- The ensemble (MLP + tuned CNN + augmented CNN) underperforms its own best member on both tasks — Task A 0.871 vs 0.878, Task B 0.604 vs 0.646. A useful negative result.
- Equal-weight soft voting only helps when members have comparable accuracy and independent errors — then averaging cancels uncorrelated mistakes (variance reduction). Here the members are unequal: the MLP (A 0.826, B 0.534) is much weaker than the two CNNs, so averaging in its confident-but-wrong probabilities drags the mean below the strong members. The two CNNs also share an architecture, so their errors are correlated — little independent signal to average away.
- The damage concentrates on the weak class — others collapses to F1 0.212 (recall 0.139), worse than the augmented CNN's 0.453, because the MLP's poor others probabilities pollute the vote. epithelial stays strong (0.900), confirming the ensemble only hurts where members disagree most.

F. Model Comparison

In [ ]:
```
# Section F: consolidate every validation result into one comparison table +
# Each model's evaluate() call appended a row to `results` (task, model, spli
# accuracy, macro_f1), so the comparison is assembled automatically.
import pandas as pd

res = pd.DataFrame(results)
# Guard against re-runs that appended duplicate rows: keep the latest per mod
res = res.drop_duplicates(subset=["task", "model", "split"], keep="last")
```

```
TARGETS    = {"A": 0.90, "B": 0.60}
TASK_TITLE = {"A": "Task A — isCancerous (binary)",
              "B": "Task B — cellType (4-class)"}

for task in ["A", "B"]:
    sub = (res[(res.task == task) & (res.split == "val")]
           .sort_values("macro_f1", ascending=False)
           .reset_index(drop=True))
    sub["meets_target"] = np.where(sub.macro_f1 >= TARGETS[task], "yes", "")
    print(f"\n=== {TASK_TITLE[task]}   (target macro-F1 >= {TARGETS[task]:.2f
    print(sub[["model", "accuracy", "macro_f1", "meets_target"]].to_string(in

# Grouped horizontal bar chart: validation macro-F1 per model, one panel per
# with the assignment target drawn as a reference line.
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, task in zip(axes, ["A", "B"]):
    sub = (res[(res.task == task) & (res.split == "val")]
           .sort_values("macro_f1"))
    bars = ax.barh(sub.model, sub.macro_f1, color="steelblue")
    ax.axvline(TARGETS[task], color="red", ls="--",
               label=f"target {TARGETS[task]:.2f}")
    ax.set_xlim(0, 1); ax.set_xlabel("validation macro-F1")
    ax.set_title(TASK_TITLE[task]); ax.legend(loc="lower right")
    for b, v in zip(bars, sub.macro_f1):
        ax.text(v + 0.01, b.get_y() + b.get_height() / 2,
                f"{v:.3f}", va="center", fontsize=9)
plt.tight_layout()
plt.show()
```

```
=== Task A — isCancerous (binary)   (target macro-F1 >= 0.90) ===
                model  accuracy  macro_f1 meets_target
  CNN + augmentation    0.8861    0.8782
Ensemble (soft vote)    0.8787    0.8706
                 CNN    0.8759    0.8669
                 MLP    0.8355    0.8257
 Logistic Regression    0.8083    0.7988
       Decision Tree    0.7423    0.7292

=== Task B — cellType (4-class)   (target macro-F1 >= 0.60) ===
                model  accuracy  macro_f1 meets_target
  CNN + augmentation    0.7079    0.6464          yes
                 CNN    0.7238    0.6347          yes
Ensemble (soft vote)    0.7108    0.6041          yes
                 MLP    0.6412    0.5339
 Logistic Regression    0.5612    0.4917
       Decision Tree    0.4950    0.4267
[saved] plots/model_comparison.png
```



- On both tasks the ranking runs Decision Tree, Logistic Regression, MLP, then CNN, each step adding the ability to model the non-linear, local pixel structure that the EDA showed

is not linearly separable. Past that point capacity stops mattering.

- Augmentation produces the best model on both tasks (Task A 0.878, Task B 0.646), not by lifting the ceiling but by curing the CNN's overfitting, which is a regularisation effect rather than a gain in capacity.
- The ensemble, the most complex option, loses to its own best member on both tasks because its weak MLP member pulls the equal-weight vote down.
- The final choice for both tasks is therefore the augmented CNN: strongest on validation, a single interpretable model rather than a three-model committee, and the committee bought nothing.

G. Final Model Selection, Justification & Evaluation

_Pick final model per task, evaluate on held-out **test patients**, report macro-F1 vs targets.

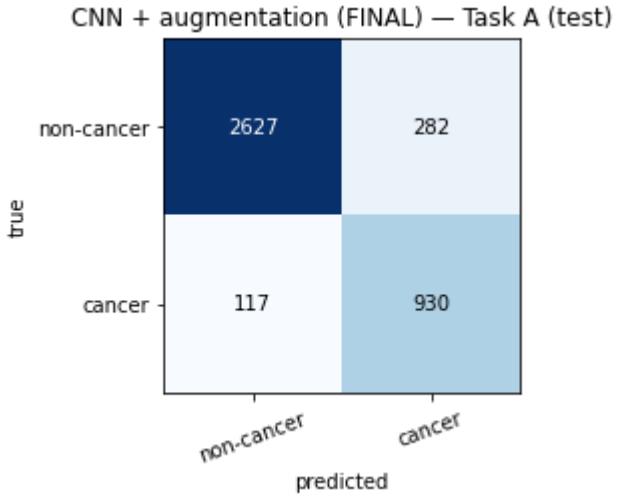
- **Model selection.** Section F's validation comparison selects the **augmentation-regularised CNN** as the final model for *both* tasks because it reached the highest validation macro-F1 (Task A 0.878, Task B 0.646), beating the plain CNN and the ensemble.
- We now evaluate this model **once** on the held-out **test patients**, a patient-disjoint set unseen during all training, tuning, and model selection. Because the test set is touched only here, the reported figures are an unbiased estimate of performance on genuinely new patients.

In [36]:

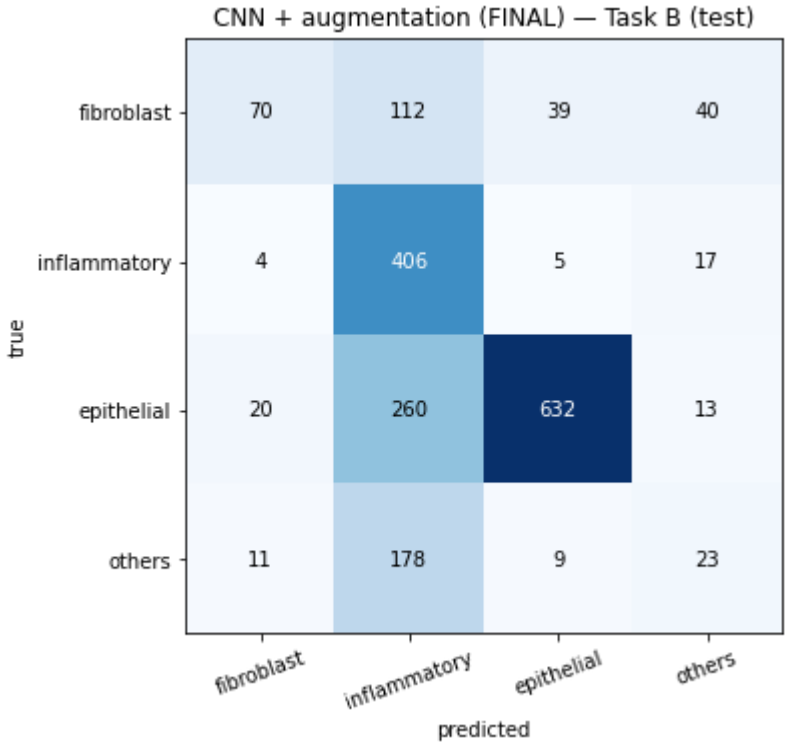
```
# Section G: final evaluation on the held-out TEST patients (touched exactly
# Selected model for BOTH tasks: the augmentation-regularised CNN — best vali
# macro-F1 in Section F. Test images are normalised with the TRAIN channel st
# no statistic is ever fitted on the test set (prevents leakage).
import pandas as pd

final_eval = {
    "A": (cnnA_aug, make_image_loader(XA, testA_idx, yA, CH_MEAN,   CH_STD,
                                      batch_size=256, shuffle=False), yA_te,
    "B": (cnnB_aug, make_image_loader(XB, testB_idx, yB, CH_MEAN_B, CH_STD_B,
                                      batch_size=256, shuffle=False), yB_te,
}

for task, (model, test_loader, y_te, names) in final_eval.items():
    evaluate("CNN + augmentation (FINAL)", task, y_te,
             nn_predict(model, test_loader), names,
             split="test", save=f"cm_{task}_final_test")

# Validation vs test macro-F1 for the chosen model (generalisation-gap check)
chosen = pd.DataFrame(results)
chosen = chosen[chosen.model.str.contains("augmentation")]
gap = (chosen.pivot_table(index="task", columns="split", values="macro_f1")
             .reindex(columns=["val", "test"]))
gap["val - test"] = (gap["val"] - gap["test"]).round(4)
TARGETS = {"A": 0.90, "B": 0.60}
gap["target"] = gap.index.map(TARGETS)
gap["test meets target"] = gap["test"] >= gap["target"]
print("\nFinal model (augmented CNN) — validation vs held-out test:")
print(gap.round(4).to_string())
```

```
[Task A] CNN + augmentation (FINAL) (test): accuracy=0.899 | macro-F1=0.876
              precision    recall  f1-score   support
```

```
              precision    recall  f1-score   support

   non-cancer      0.957     0.903     0.929      2909
       cancer      0.767     0.888     0.823      1047

     accuracy                          0.899      3956
    macro avg      0.862     0.896     0.876      3956
 weighted avg      0.907     0.899     0.901      3956

[saved] plots/cm_A_final_test.png
```




```
[Task B] CNN + augmentation (FINAL) (test): accuracy=0.615 | macro-F1=0.475
              precision    recall  f1-score   support

  fibroblast      0.667     0.268     0.383       261
inflammatory      0.425     0.940     0.585       432
  epithelial      0.923     0.683     0.785       925
      others      0.247     0.104     0.146       221

    accuracy                          0.615      1839
   macro avg      0.565     0.499     0.475      1839
weighted avg      0.688     0.615     0.604      1839

[saved] plots/cm_B_final_test.png
```




```
Final model (augmented CNN) — validation vs held-out test:
split     val    test  val - test  target  test meets target
task
```

```
A       0.8782  0.8764      0.0018      0.9             False
B       0.6464  0.4748      0.1716      0.6             False
```

- The two tasks bahave differently on held-out patients.
- Task A generalises cleanly: val 0.878 → test 0.876, a 0.002 gap, with 90% accuracy and 89% cancer recall. It is still under the 0.90 target, but this is a real ceiling, not a tuning miss — at 27×27 some cells are genuinely ambiguous, and the EDA showed cancer and non-cancer overlap in pixel space, so a small error remains.
- Task B does not transfer: val 0.646 → test 0.475, a 0.17 gap that misses the 0.60 target. This is the patient-level split working as intended — cell-type appearance varies enough between patients that a model tuned on one set that does not carry to new ones. On test it floods inflammatory (recall 0.94, precision 0.42) and collapses on rare classes (others F1 0.15, fibroblast recall 0.27). The validation figure was optimistic, 0.475 is the honest estimate for a new patient. Task B needs more patients and class-balanced training, not more model capacity.